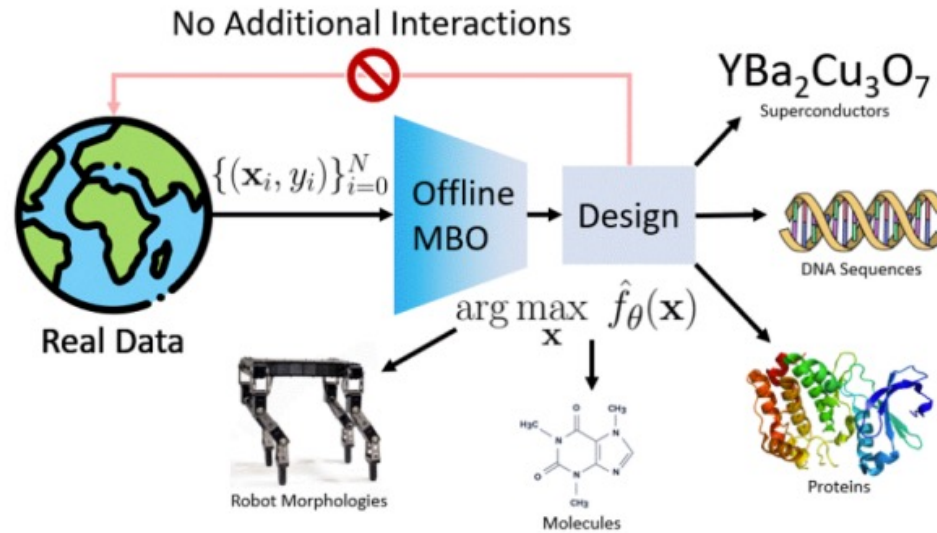


IE437 Data-Driven Decision Making and Control
2024 Spring Semester

6. Data-Driven Design Optimization (Generative Model Based)

Jinkyoo Park
Associate Professor
Industrial and Systems Engineering Department

Motivation (Recap)



- In real world optimization problems, evaluation of objective function is **expensive** and **prohibitive** (Examples) Molecule Design, Automated Aircraft Design, ...
- Therefore, constructing a data-driven model given a fixed, offline dataset is crucial
- Formally, we try to find an input $\mathbf{x}$ such that

$$\text{find } \mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmax}} f(\mathbf{x}) \text{ with **only** a fixed dataset, } D = \{(\mathbf{x}_1, f(\mathbf{x}_1)), \dots, (\mathbf{x}_N, f(\mathbf{x}_N))\}$$

Motivation

- Formally, we try to find an input $\mathbf{x}$ such that

$$\text{find } \mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmax}} f(\mathbf{x}) \text{ with } \mathbf{only} \text{ a fixed dataset, } D = \{(\mathbf{x}_1, f(\mathbf{x}_1)), \dots, (\mathbf{x}_N, f(\mathbf{x}_N))\}$$

- Main Approaches
 - Surrogate model-based:** Approximate $f(x)$ using previously collected data, choose the query that maximizes the surrogate model
 - Generative model-based:** Learn an inverse function $f^{-1}(y)$ using previously collected data, choose the query generated from the inverse function given by a desired output.
- Today, we'll focus on Generative model-based approaches

Motivation

- **Main Problem of Offline Model-Based Optimization:**
 - Input space is high-dimensional, but usually with narrow manifold for valid inputs (Protein Design, Natural Image, ...)
 - Since training distribution shows a small fraction of input space, optimizer can easily fall into adversarial inputs which results in arbitrary outputs
- **Surrogate model based approach resolve these issues by “Conservatism”**
 - Limit the expressivity of the model and often fails to find a better design
- **Generative model based approach samples the distribution conditioned on the desired properties**

Jointly model $p(\mathbf{x}, y) = p(y|\mathbf{x})p(\mathbf{x})$

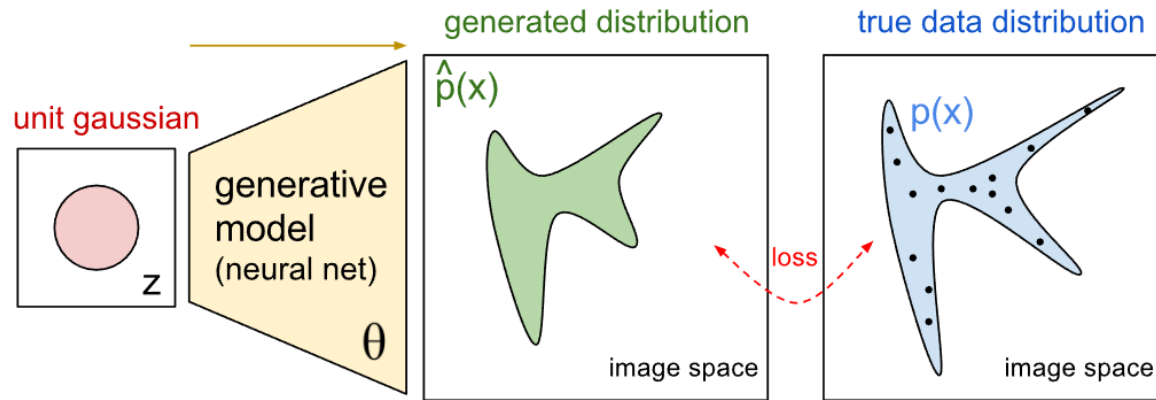
Sample from $p(\mathbf{x}|y \geq y_{max})$, more generally, $p(\mathbf{x}|S)$

- Avoid solving optimization problem (i.e., gradient decent) but just sample (convenient)
- Can provide a diversified designs (**high expressivity of generative models**, i.e., multi-modality)
- Implicitly satisfy the constraints while sampling (trained only by the feasible designs)

Generative Model Overview

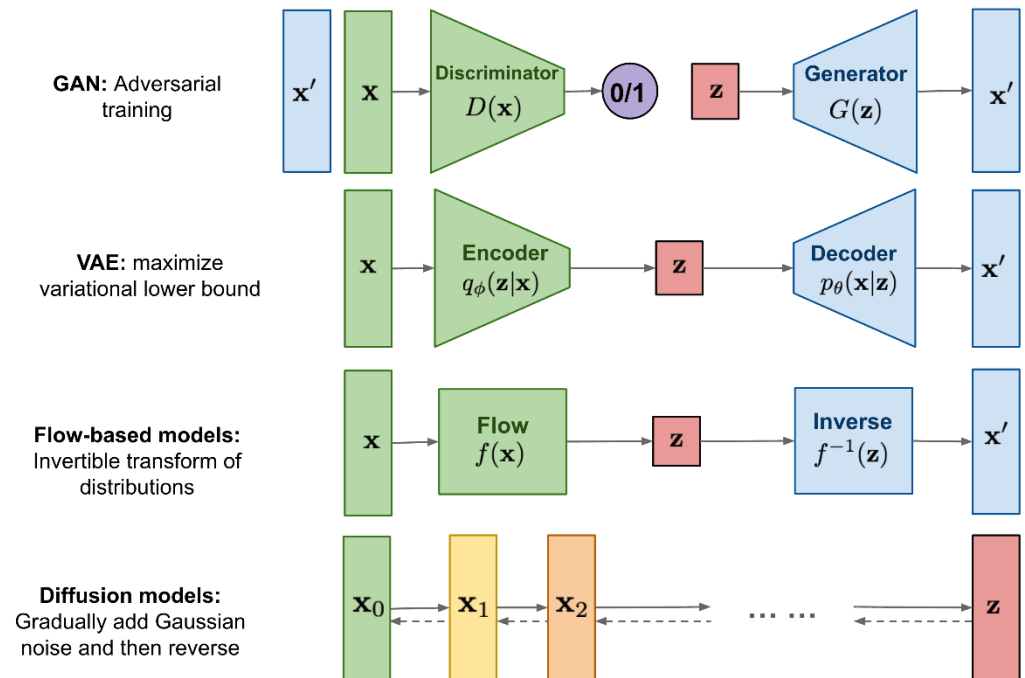
Generative Models

GOAL: Train $p_{\theta}(x)$ with $\{\mathbf{x}_i\}_{i=1}^N \sim p_{data}(x)$

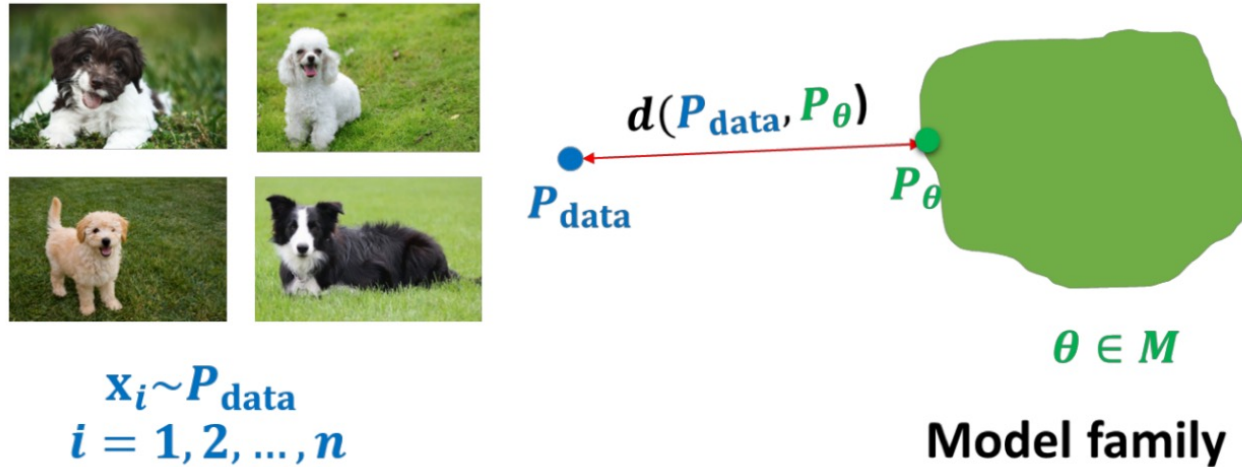


Many generative models:

1. Variational Auto-Encoders (VAE)
2. Autoregressive models
3. Normalizing flows
4. Energy-based models
5. Generative Adversarial Networks
6. Diffusion Models



Variational Inference



Goal of learning a generative model is to learn p_{θ} given the model family to minimize the two distribution:

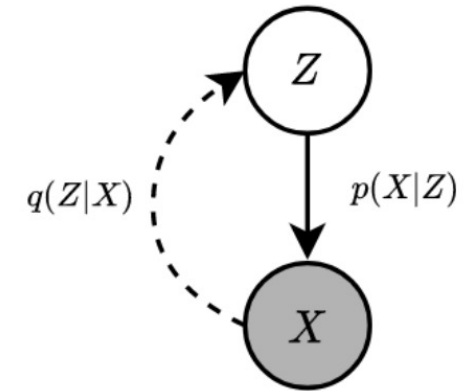
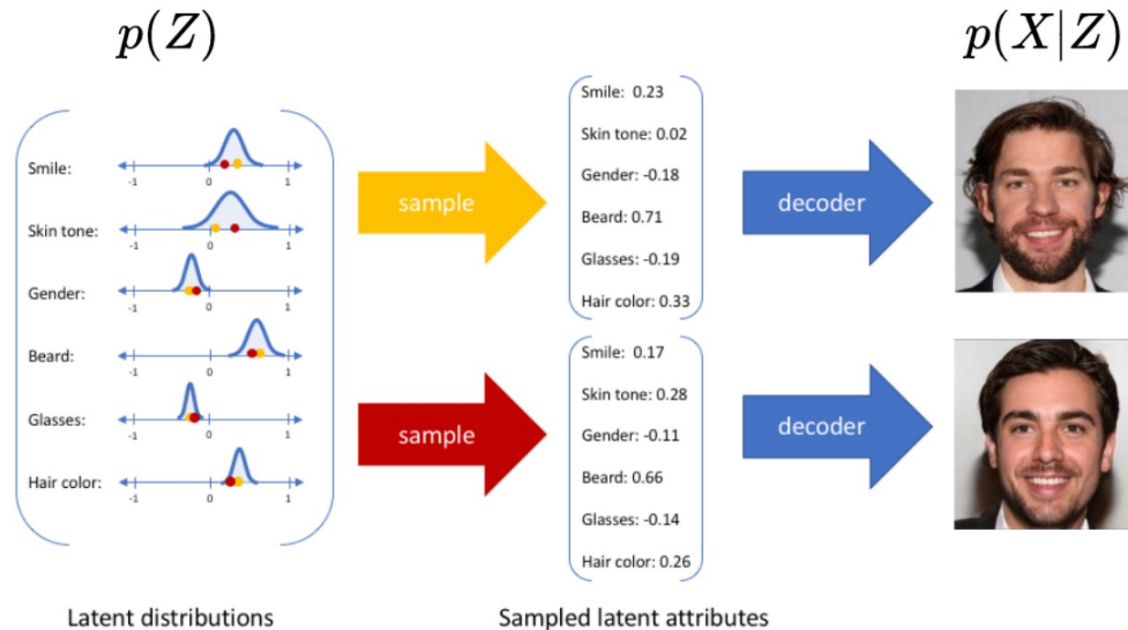
$$\arg \min_{p_{\theta}} \mathbb{D}_{KL}(p_{\text{data}} || p_{\theta}) = \max_{p_{\theta}} \frac{1}{|\mathcal{D}|} \sum_{x_i \in \mathcal{D}} \log p_{\theta}(x_i)$$

$$\mathbb{D}_{KL}(P || Q) = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right]$$

The KL divergence is the measure of inefficiency in using the probability distribution Q to approximate the true probability distribution P

Variational Inference

Learning the likelihood $p_{\theta}(x_i)$ of high-dimensional x_i is difficult.

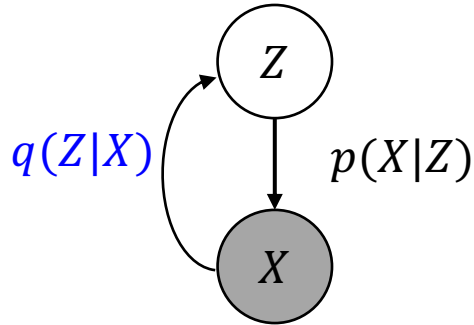


Construct a latent factor model:

$$p(X) = \int_Z P(X|Z)P(Z)dZ$$

- Data is governed by a simple latent distribution $p(Z)$.
- The observed data X is generated by a conditional distribution $p(X|Z)$.

Variational Inference (Evidence Perspective)



Maximizing log likelihood $\log p(X)$ (Evidence)

=

Maximizing ELBO w.t $q(Z|X)$

=

Minimizing $\mathbb{D}_{KL}(q(Z|X)||p(Z|X))$ w.t $q(Z|X)$

$$\log p(X) = \log \int_Z p(X, Z) dZ$$

$$= \log \int_Z \frac{q(Z|X)}{q(Z|X)} p(X, Z) dZ$$

$$= \log \int_Z q(Z|X) \frac{p(X, Z)}{q(Z|X)} dZ$$

$$= \log \mathbb{E}_{Z \sim q(Z|X)} \left[\frac{p(X, Z)}{q(Z|X)} \right]$$

$$\geq \mathbb{E}_{Z \sim q(Z|X)} \left[\log \frac{p(X, Z)}{q(Z|X)} \right] \quad \because \text{Jensen's Inequality}$$

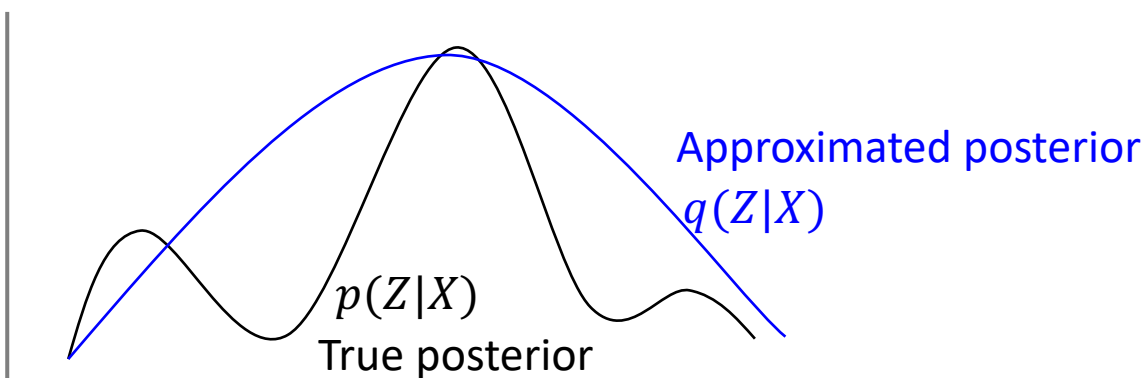
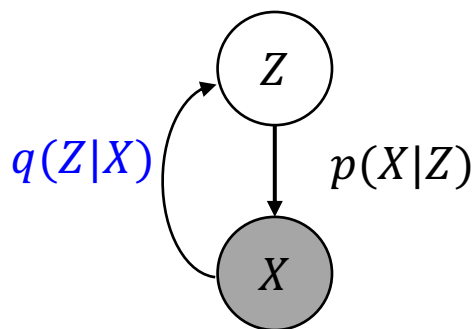
$$= \mathbb{E}_{Z \sim q(Z|X)} \left[\log \frac{p(X|Z)P(Z)}{q(Z|X)} \right]$$

$$= \mathbb{E}_{Z \sim q(Z|X)} [\log p(X|Z)] + \mathbb{E}_{Z \sim q(Z|X)} [\log p(Z) - \log q(Z|X)]$$

$$= \mathbb{E}_{Z \sim q(Z|X)} [\log p(X|Z)] - \mathbb{D}_{KL}(q(Z|X)||p(Z))$$

$$= \text{Evidence Lower Bound (ELBO)}$$

Variational Inference (Optimization Perspective)



Maximizing log likelihood $\log p(X)$ (Evidence)

=

Maximizing **ELBO** w.t $q(Z|X)$

=

Minimizing $\mathbb{D}_{KL}(q(Z|X)||p(Z|X))$ w.t $q(Z|X)$

$$\begin{aligned}
 \mathbb{D}_{KL}(q(Z|X)||p(Z|X)) &= \int_Z q(Z|X) \log \frac{q(Z|X)}{p(Z|X)} dZ \\
 &= \int_Z q(Z|X) \log \frac{q(Z|X)p(X)}{p(Z, X)} dZ \\
 &= \int_Z q(Z|X) \log \frac{q(Z|X)}{p(Z, X)} dZ + \int_Z q(Z|X) \log p(X) dZ \\
 &= \mathbb{E}_{z \sim q(Z|X)} \left[\log \frac{q(Z|X)}{p(Z, X)} dZ \right] + \mathbb{E}_{z \sim q(Z|X)} [\log p(X)] \\
 &= -\mathbb{E}_{z \sim q(Z|X)} \left[\log \frac{p(Z, X)}{q(Z|X)} dZ \right] + \mathbb{E}_{z \sim q(Z|X)} [\log p(X)] \\
 &= -\mathbf{ELBO} + \mathbb{E}_{z \sim q(Z|X)} [\log p(X)]
 \end{aligned}$$

$$\mathbb{D}_{KL}(q(Z|X)||p(Z|X)) = -\mathbf{ELBO} + \mathbb{E}_{z \sim q(Z|X)} [\log p(X)] \geq 0$$

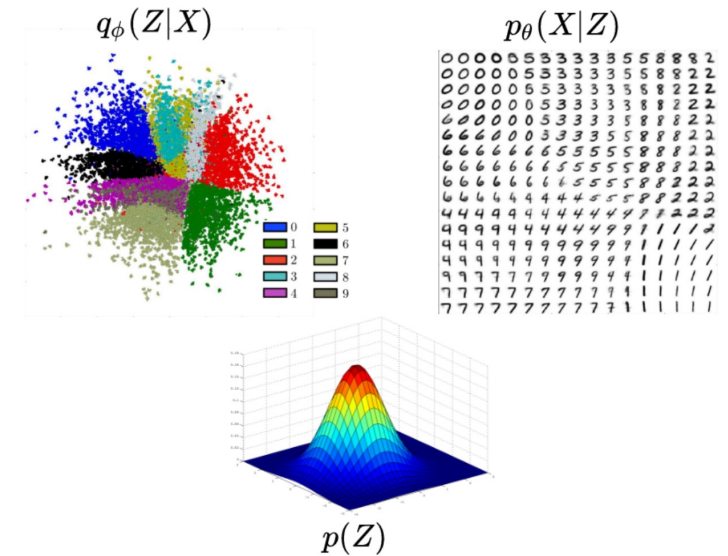
$$\mathbb{E}_{z \sim q(Z|X)} [\log p(X)] \geq \mathbf{ELBO}$$

Variational Autoencoders

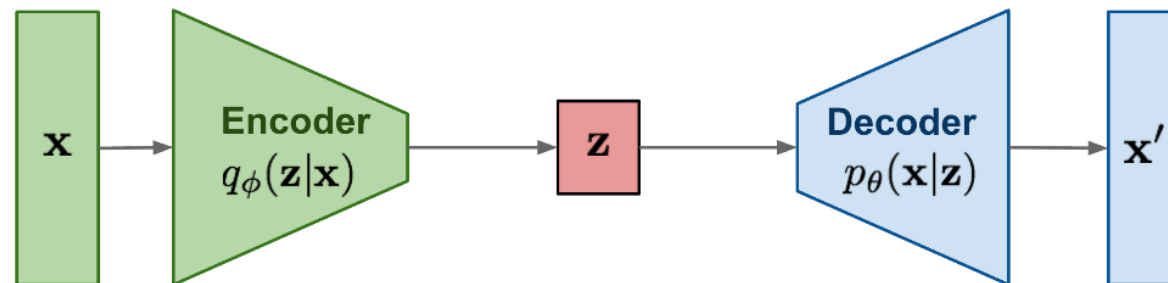
- Variational Autoencoders (VAEs)
 - One of the most widely used Generative Model
 - Consists of probabilistic encoder and decoder network
 - **Generative Process**) Sample $z \sim \mathcal{N}(0, I)$, generate $x = p_\theta(x|z)$
 - **Inference Process**) Using [Variational Inference](#)

Optimal approximate posterior:

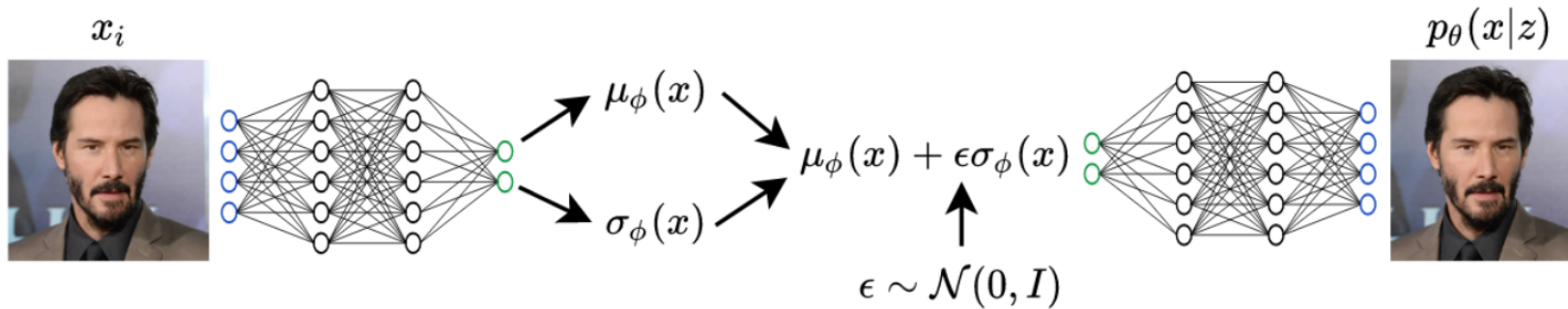
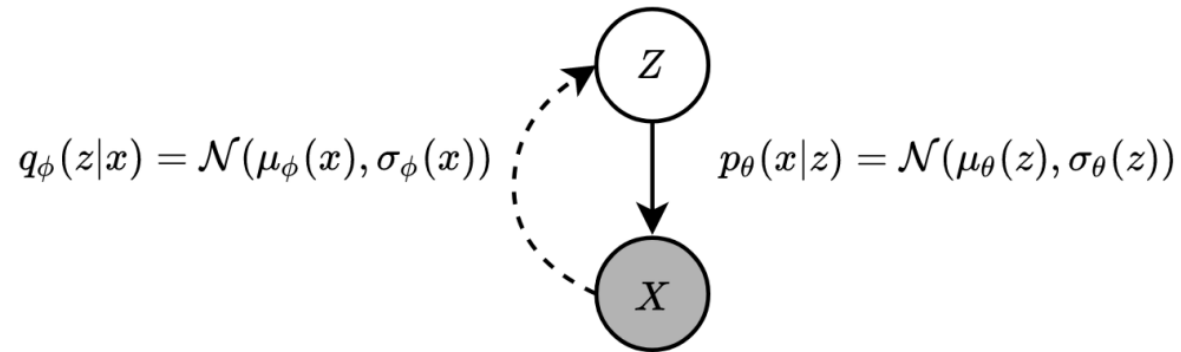
$$\begin{aligned} q_{\phi}^*(z|x) &= \operatorname{argmax}_{\phi} \mathbb{E}_{z \sim q(Z|X)} \left[\log \frac{p_{\theta}(X,Z)}{q_{\phi}(z|x)} \right] \\ &= \operatorname{argmax}_{\phi} \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - \text{KL}[q_{\phi}(z|x) || p_{\theta}(z)] \end{aligned}$$



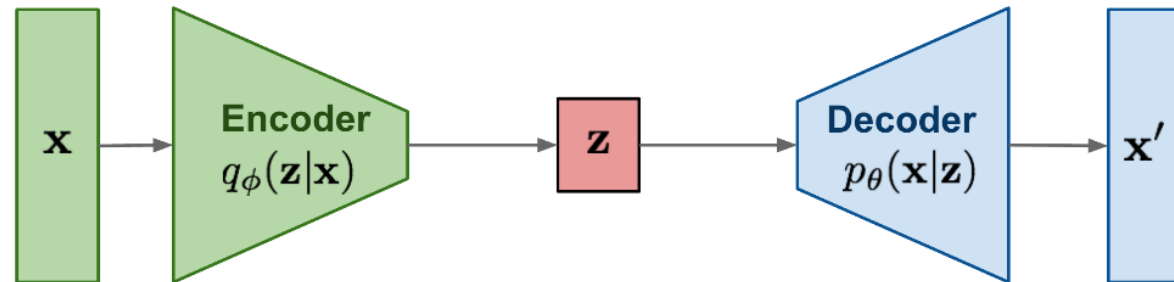
VAE: maximize
variational lower bound



Variational Autoencoders



VAE: maximize
variational lower bound

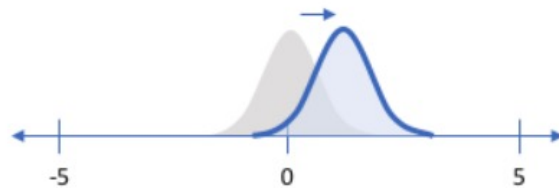


$$\max_{\phi, \theta} \frac{1}{N} \sum_{i=1}^N \log p_\theta(x_i | \mu_\phi(x) + \epsilon \sigma_\phi(x)) - \mathbb{D}_{KL}[\mathcal{N}(\mu_\phi(x_i), \sigma_\phi(x_i)) || \mathcal{N}(0, I)]$$

Variational Autoencoders

$$\max_{\phi, \theta} = \underbrace{\mathbb{E}_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)]}_{\text{reconstruction}} - \underbrace{\mathbb{D}_{KL}(q_{\phi}(z|x) || p(z))}_{\text{KL regularization}}$$

Penalizing reconstruction loss encourages the distribution to describe the input



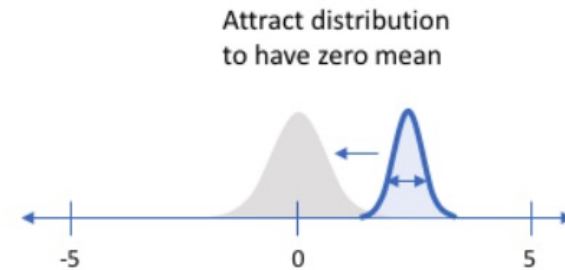
Our distribution deviates from the prior to describe some characteristic of the data

Without regularization, our network can “cheat” by learning narrow distributions



With a small enough variance, this distribution is effectively only representing a single value

Penalizing KL divergence acts as a regularizing force



Ensure sufficient variance to yield a smooth latent space

The VAE objectives arranges data on a compact manifold (we can sample from) in a continuous smooth way.

Conditional Variational Auto Encoder

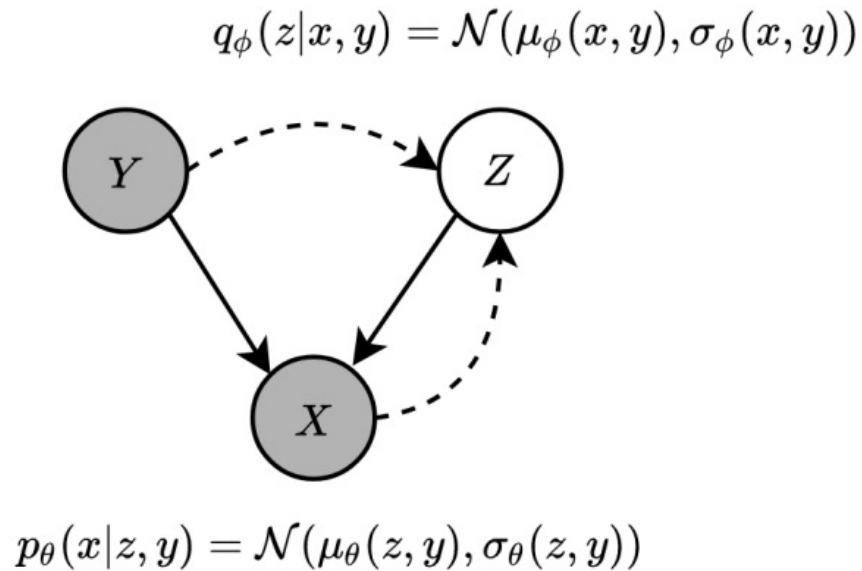


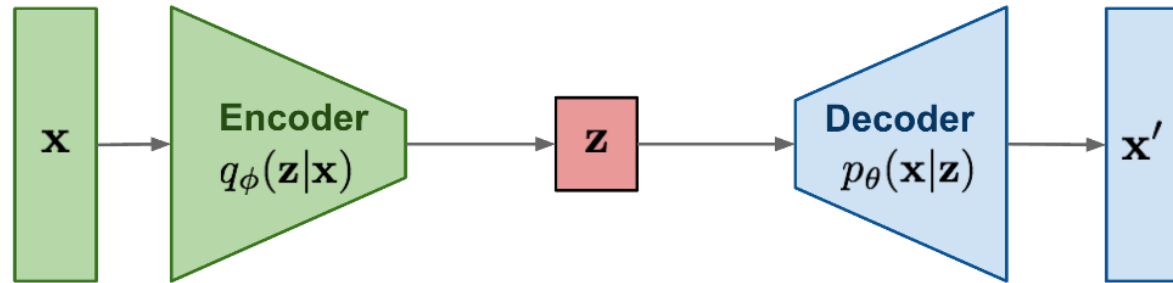
Figure: Samples from CVAE trained on SVHN

$$\max_{\theta, \phi} \mathbb{E}_{z \sim q_\phi(z|x, y)} [\log p_\theta(x|z, y)] - \mathbb{D}_{KL}(q_\phi(z|x, y) || p_\theta(z|y))$$

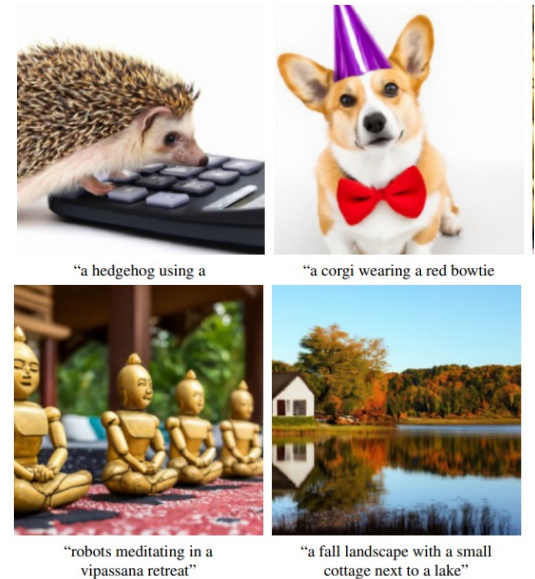
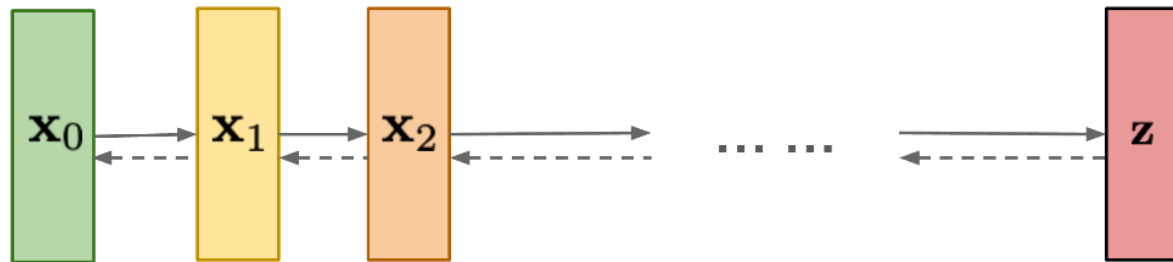
- We just need to input the condition y into the encoder and decoder networks.
- This conditionalization will play an important role in inverse design:
 - y can be specified to be the desired properties
 - x can be the corresponding design

Diffusion Models

VAE: maximize variational lower bound



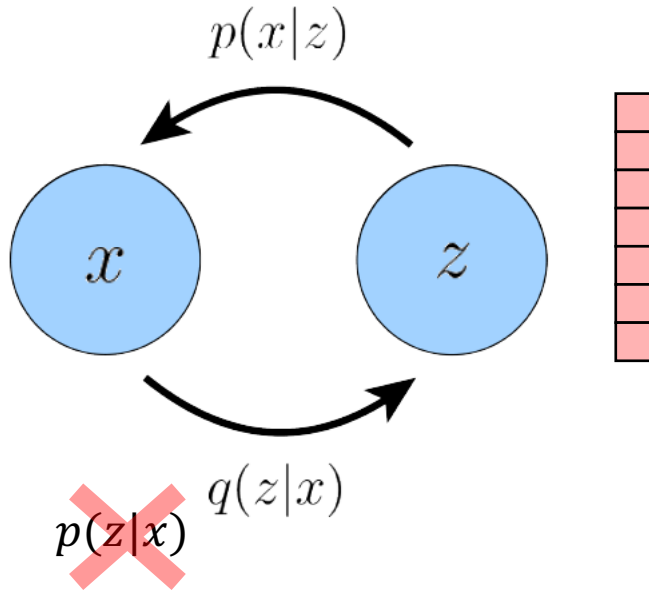
Diffusion models:
Gradually add Gaussian noise and then reverse



Advantages of the diffusion model over the VAE:

- **More flexible distribution modeling:** VAEs typically assume that the distribution of the latent variables is Gaussian, which may not be flexible enough to capture complex data distributions.
- **Better sample quality:** The diffusion model can generate high-quality samples with high resolution and realistic details, while VAEs may suffer from blurry or low-resolution samples. This is because the diffusion model uses a series of transformations to gradually refine the samples, which leads to more accurate reconstructions.
- **No need for an explicit encoder:** The diffusion model does not require an explicit encoder to map the data to the latent space, unlike the VAE. This can make the diffusion model easier to train and faster to generate samples, as there is no need to optimize the encoder network.

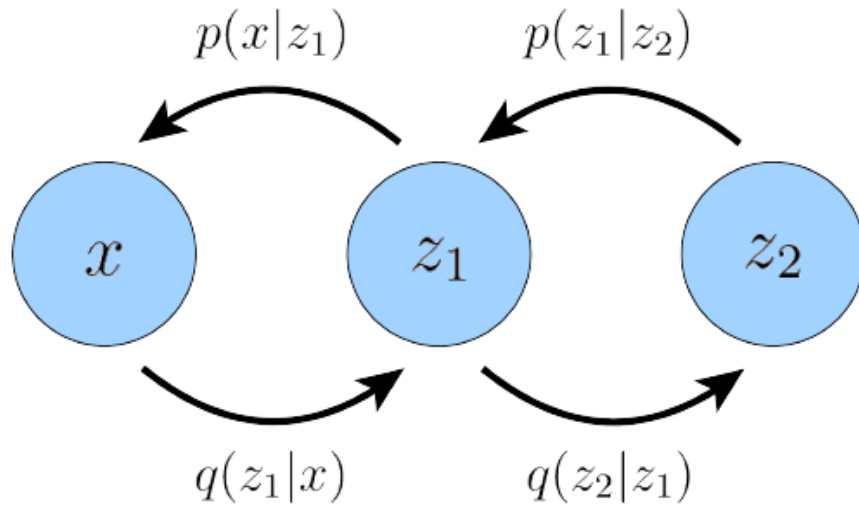
Diffusion Models: From the perspective of VAEs



ELBO Objective

$$L_{\theta, \phi}(x) = E_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)} \right]$$

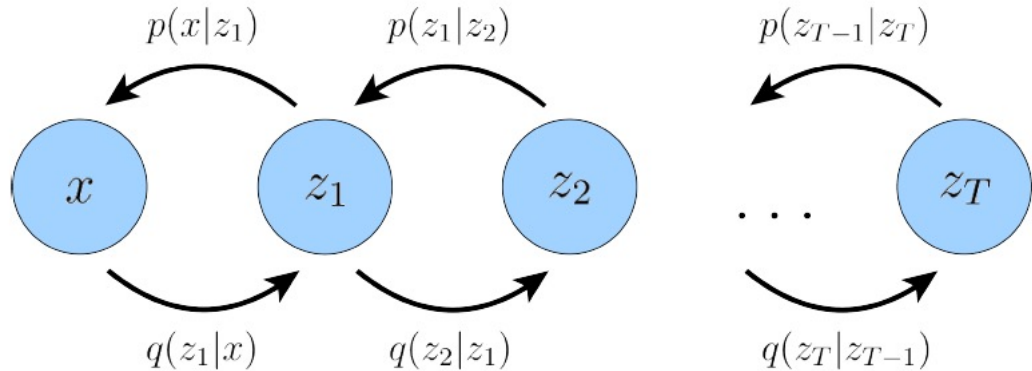
Diffusion Models: From the perspective of VAEs



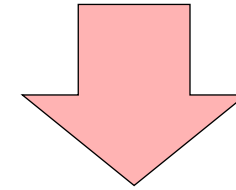
ELBO Objective

$$L_{\theta, \phi}(x) = E_{q_{\phi}(z_1, z_2 | x)} \left[\log \frac{p_{\theta}(x, z_1, z_2)}{q_{\phi}(z_1, z_2 | x)} \right]$$

Diffusion Models: VAE Perspective



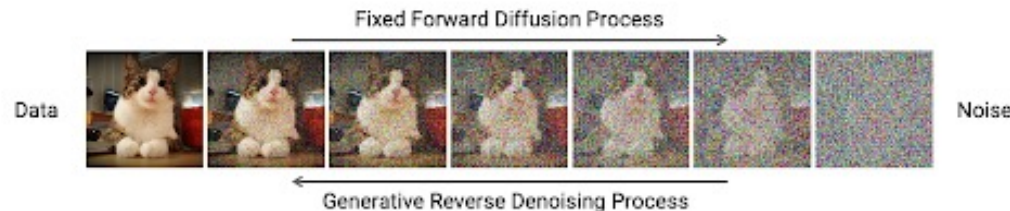
$$L_{\theta, \phi}(x) = E_{q_{\phi}(z_{1:T}|x)} \left[\log \frac{p_{\theta}(x, z_{1:T})}{q_{\phi}(z_{1:T}|x)} \right]$$



Key differences with VAE:

1. Latent dimension = data dimension
2. Latent encoder is pre-defined linear Gaussian model (no parameter to train)
3. Set $p(x_T) = N(0, I)$

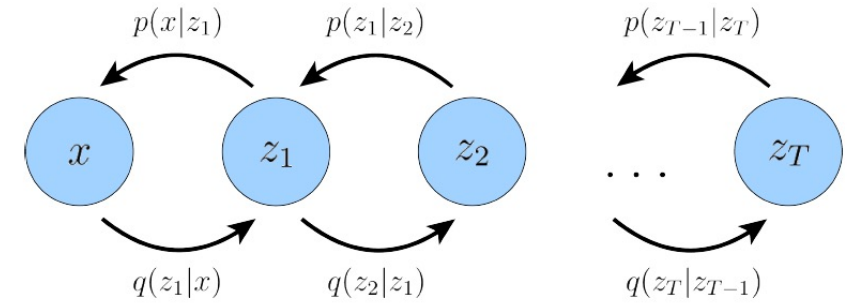
$$L_{\theta}(x) = E_{q(z_{1:T}|x)} \left[\log \frac{p_{\theta}(x_0, x_{1:T})}{q(x_{1:T}|x_0)} \right]$$



Diffusion Models: VAE Perspective

- Forward trajectory

$$q(x_{1:T}|x_0) = \prod_{t=1}^T q(x_t|x_{t-1})$$
$$x_0 \sim q(x_0)$$



- Reverse trajectory

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t)$$

- Objective

$$L_\theta(x) = E_{q(x_{1:T}|x)} \left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right]$$

$$L_\theta(x) = E_{q(x_1|x_0)} [\log p_\theta(x_0|x_1)] - D_{KL}(q(x_T|x_0), p(x_T)) - \sum_{t=2}^T E_{q(x_t|x_0)} [D_{KL}(q(x_{t-1}|x_t, x_0), p_\theta(x_{t-1}|x_t))]$$

Diffusion Models: VAE Perspective

- Objective:

$$L_{\theta}(x) = E_{q(x_1|x_0)}[\log p_{\theta}(x_0|x_1)] - D_{KL}(q(x_T|x_0), p(x_T)) - \sum_{t=2}^T E_{q(x_t|x_0)}[D_{KL}(q(x_{t-1}|x_t, x_0), p_{\theta}(x_{t-1}|x_t))]$$

- Next question: What distribution should $q(x_t|x_{t-1})$ be?
- Key considerations for efficiency
 1. Closed form expression for $q(x_t|x_0)$
 2. Closed form expression for $q(x_{t-1}|x_t, x_0)$
 3. Limit distribution $q_{\infty} = \lim_{T \rightarrow \infty} q(x_T|x_0)$ should not depend on x_0
- The most obvious distribution that satisfies is the **Gaussian** distribution:

$$q(x_t|x_{t-1}) = N(x_t | \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

$$\beta_t \in [0,1]$$

This is the formulation introduced in DDPM!

Diffusion Models (DDPM)

1. Closed form expression for $q(x_t|x_0)$

Using reparameterization trick:

$$q(x_t|x_0) = N(x_t|\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

2. Closed form expression for $q(x_{t-1}|x_t, x_0)$ (denoising)

Using Bayes' rule:

$$\begin{aligned} q(x_{t-1}|x_t, x_0) &= N(x_{t-1}|\tilde{\mu}(x_t, x_0), \tilde{\beta}_t I) \\ \tilde{\mu}(x_t, x_0) &= \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t \\ \tilde{\beta}_t &= \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t \end{aligned}$$

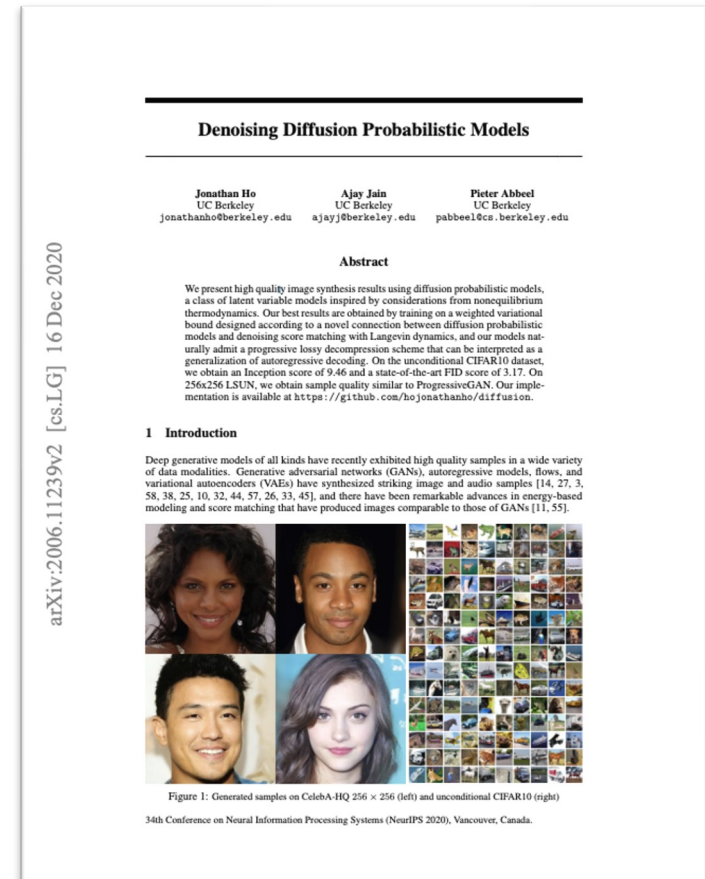
3. Limit distribution $q_\infty = \lim_{t \rightarrow \infty} q(x_t|x_0)$ should not depend on x_0

$$\lim_{t \rightarrow \infty} \bar{\alpha}_t = 0$$

$$q_\infty = \lim_{t \rightarrow \infty} q(x_t|x_0) = N(0, I)$$

Denoising Diffusion Probabilistic Models

Jonathan Ho et al., NeurIPS 2020



- Proposed clever algorithmic choices.

Diffusion Models (DDPM)

- Objective

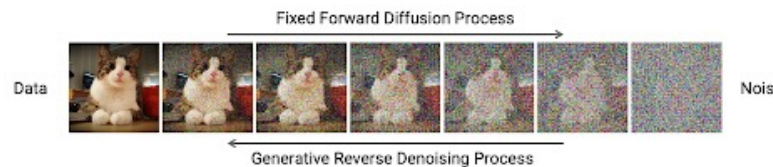
$$L_{t-1} = E_{q(x_t|x_0)} \left[\frac{1}{2\sigma_t^2} \|\tilde{\mu}(x_t, x_0) - \mu_\theta(x_t, t)\|^2 \right]$$

- Depending on the parameterization of the model, we can reformulate the objective into three other forms:

$$1. \quad L_{t-1} = E_{q(x_t|x_0)} \left[\frac{1}{2\sigma_t^2} \frac{\bar{\alpha}_{t-1}(1-\alpha_t)^2}{(1-\bar{\alpha}_t)^2} \|x_0 - \hat{x}_\theta(x_t, t)\|^2 \right] \quad (\text{from } \tilde{\mu}(x_t, x_0))$$

$$2. \quad L_{t-1} = E_{q(x_t|x_0)} \left[\frac{1}{2\sigma_t^2} \frac{(1-\alpha_t)^2}{(1-\bar{\alpha}_t)^2 \alpha_t} \|\epsilon_0 - \hat{\epsilon}_\theta(x_t, t)\|^2 \right] \quad (\text{Reparameterization})$$

$$3. \quad L_{t-1} = E_{q(x_t|x_0)} \left[\frac{1}{2\sigma_t^2} \frac{(1-\alpha_t)^2}{\alpha_t} \|\nabla_{x_t} \log p(x_t) - s_\theta(x_t, t)\|^2 \right] \quad (\text{Tweedie's Formula})$$



Diffusion Models: EBM perspective

- Energy-based models

$$p_{\theta}(x) = \frac{\exp(-\varepsilon_{\theta}(x))}{Z_{\theta}}$$

Energy function

$$Z_{\theta} = \int \exp(-\varepsilon_{\theta}(x)) dx$$

- There are a few ways to formulate the objectives for EBM :
 - ① Maximum likelihood training
 - ② Score Matching (Score Based Generative Model)
 - ③ Noise Contrastive Estimation

Diffusion Models: EBM perspective

① Maximum likelihood training

$$l(\theta) = E_{x \sim p_{data}(x)} [\log p_{\theta}(x)]$$

$$p_{\theta}(x) = \frac{\exp(-\varepsilon_{\theta}(x))}{Z_{\theta}}$$

$$\nabla_{\theta} \log p_{\theta}(x) = -\nabla_{\theta} \varepsilon_{\theta}(x) - \nabla_{\theta} \log Z_{\theta}$$

- Evaluating a normalized constant Z_{θ} is a typically intractable quantity for any general $\varepsilon_{\theta}(x)$.
- Thus to make maximum likelihood training feasible, likelihood-based models must either
 1. restrict their model architectures (e.g., causal convolutions in autoregressive models, invertible networks in normalizing flow models) to make Z_{θ} tractable,
 2. approximate the normalizing constant (e.g., variational inference in VAEs, or MCMC sampling used in contrastive divergence) which may be computationally expensive.

Diffusion Models: EBM perspective

② Score Matching

- The score function instead of the density function, we can sidestep the difficulty of intractable normalizing constants. The score function of a distribution $p(x)$ is defined as

$$\nabla_x \log p(x)$$

- The score-based model is learned such that $s_\theta(x) \approx \nabla_x \log p(x)$ and can be parameterized without worrying about the normalizing constant.
- We can easily parameterize a score-based model with the energy-based model defined in equation

$$s_\theta(x) \approx \nabla_x \log p(x) = \nabla_x \log \left(\frac{\exp(-\varepsilon_\theta(x))}{Z_\theta} \right) = -\nabla_x \varepsilon_\theta(x) - \cancel{\nabla_x \log Z_\theta}^0 = -\nabla_x \varepsilon_\theta(x)$$

- Score Matching Objective minimizes the Fischer divergence:

$$D_F(p_{data}(x), p_\theta(x)) = E_{p_{data}(x)} \left[\frac{1}{2} \|\nabla_x \log p_{data}(x) - \underset{s_\theta(x)}{\nabla_x \log p_\theta(x)}\|^2 \right]$$

Diffusion Models: EBM perspective

② Score Matching

- Training objective

$$\frac{1}{2} \mathbb{E}_{p_{data}(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p_{data}(\mathbf{x}) - s_{\theta}(\mathbf{x})\|_2^2]$$

- Directly computing this divergence, however, is infeasible because it requires access to the unknown data score $\nabla_{\mathbf{x}} \log p_{data}(\mathbf{x})$. Fortunately, there exists a family of methods called score matching

- Which is equivalent to (Hyvärinen, 2005)

$$\mathbb{E}_{p_{data}(\mathbf{x})} \left[\text{tr}(\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x})) + \frac{1}{2} \|s_{\theta}(\mathbf{x})\|_2^2 \right]$$

- If the data is perturbed with a noise distribution $q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x})$, it is equivalent to (Vincent, 2011)

$$\frac{1}{2} \mathbb{E}_{q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x})p_{data}(\mathbf{x})} [\|\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}|\mathbf{x}) - s_{\theta}(\tilde{\mathbf{x}})\|_2^2]$$

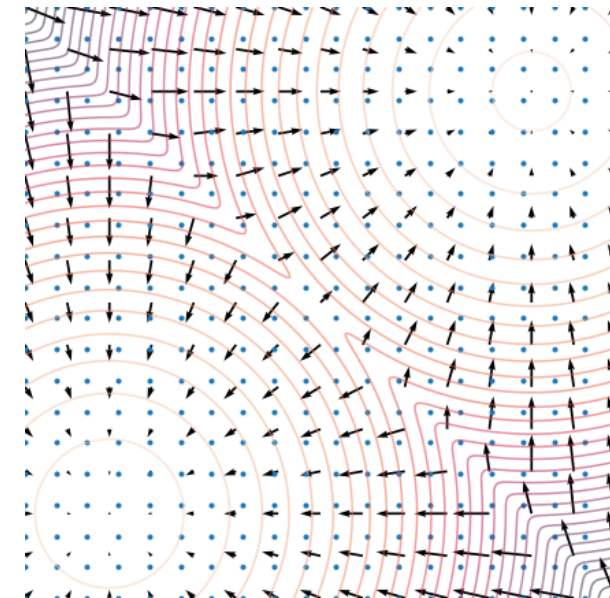
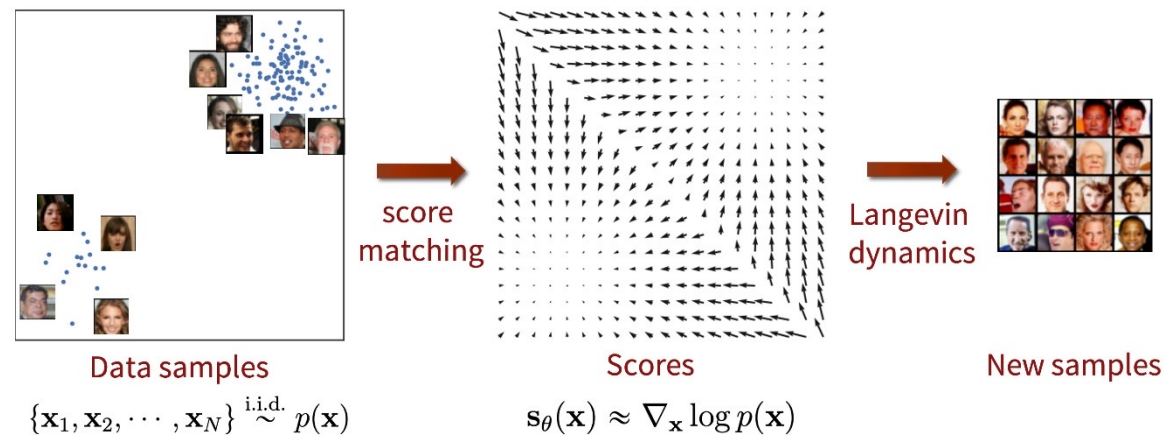
Diffusion Models: EBM perspective

② Score Matching

- After Score function $s_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$ is estimated, we can use an iterative procedure called Langevin dynamics to draw samples from it.
- Langevin dynamics provides an MCMC procedure to sample from a distribution $p(\mathbf{x})$ using only its score function $\nabla_{\mathbf{x}} \log p(\mathbf{x})$. Specifically, it initializes the chain from an arbitrary prior distribution $x_0 \sim \pi(\mathbf{x})$ and then iterates the following

$$\tilde{x}_t = \tilde{x}_{t-1} + \frac{\epsilon}{2} \nabla_{\mathbf{x}} \log p(\tilde{x}_{t-1}) + \sqrt{\epsilon} z_t, \quad z_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

- ✓ When $\epsilon \rightarrow 0$ and $T \rightarrow \infty$, the distribution of $\tilde{x}_T$ equals $p(\mathbf{x})$ (M. Welling 2011)

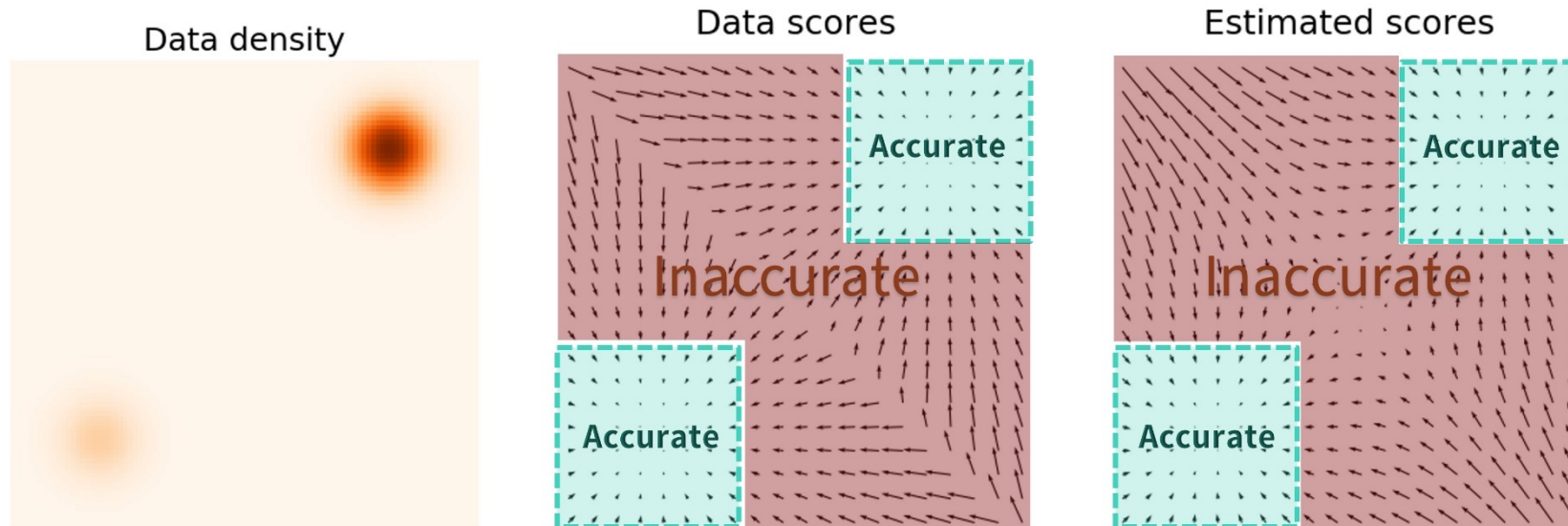


Naive score-based generative modeling and its pitfalls

- The key challenge is the fact that the estimated score functions are inaccurate in low density regions, where few data points are available for computing the score matching objective.

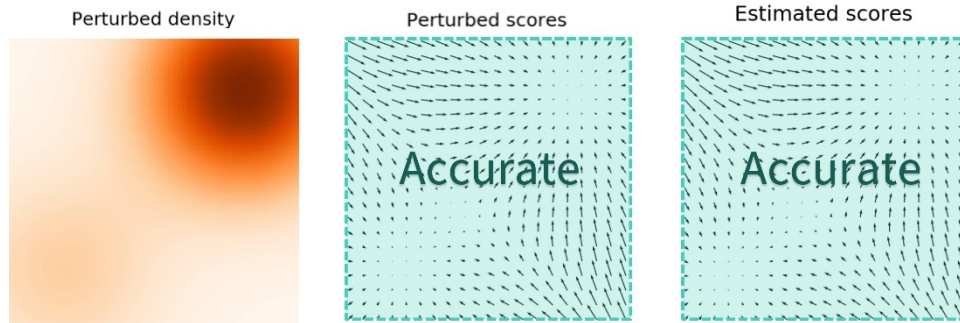
$$\mathbb{E}_{p(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x})\|_2^2] = \int p(\mathbf{x}) \|\nabla_{\mathbf{x}} \log p(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x})\|_2^2 d\mathbf{x}.$$

- Since the l_2 differences between the true data score function and score-based model are weighted by $p(\mathbf{x})$, they are largely ignored in low density regions where $p(\mathbf{x})$ is small.



Score-based generative modeling with multiple noise perturbations

- Solution: **perturb data points with noise and train score-based models on the noisy data points instead.**



- ✓ When the noise magnitude is sufficiently large, it can populate low data density regions to improve the accuracy of estimated scores.
- How do we choose an appropriate noise scale for the perturbation process? (**multiple scales of noise perturbations**)

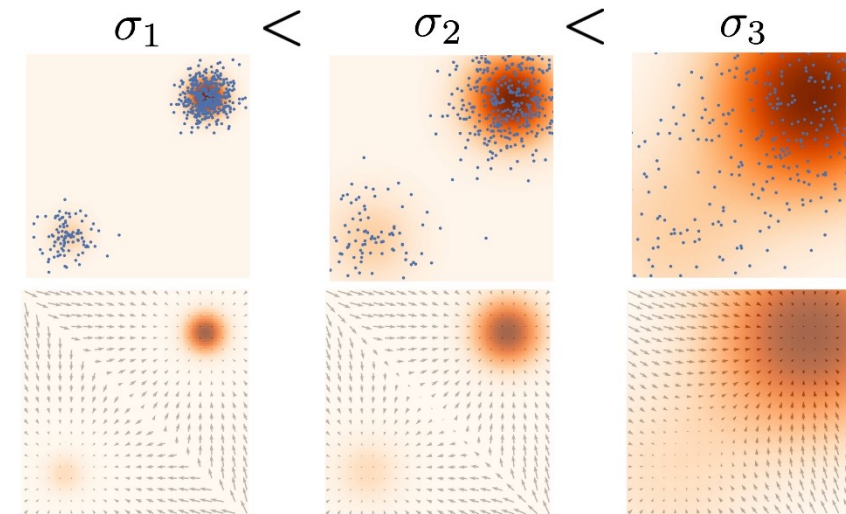
$$p_{\sigma_i}(\mathbf{x}) = \int p(\mathbf{y}) \mathcal{N}(\mathbf{x}; \mathbf{y}, \sigma_i^2 I) d\mathbf{y}.$$

$$\mathbf{s}_{\theta}(\mathbf{x}, i) \approx \nabla_{\mathbf{x}} \log p_{\sigma_i}(\mathbf{x}) \quad \text{Noise Conditional Score-Based Model}$$

$$\sum_{i=1}^L \lambda(i) \mathbb{E}_{p_{\sigma_i}(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p_{\sigma_i}(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x}, i)\|_2^2],$$

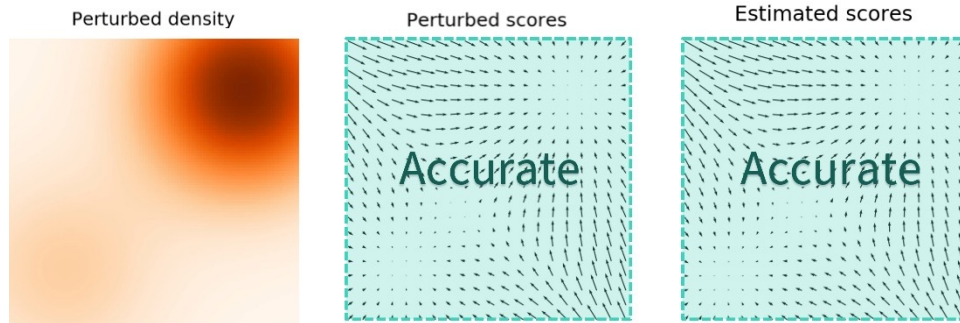
$$\lambda(i) = \sigma_i^2$$

Parameterize the score-based model $\mathbf{s}_{\theta}(\mathbf{x}, i)$ with U-Net skip connections



Score-based generative modeling with multiple noise perturbations

- Solution: **perturb data points with noise and train score-based models on the noisy data points instead.**



- ✓ When the noise magnitude is sufficiently large, it can populate low data density regions to improve the accuracy of estimated scores.
- How do we choose an appropriate noise scale for the perturbation process? (**multiple scales of noise perturbations**)

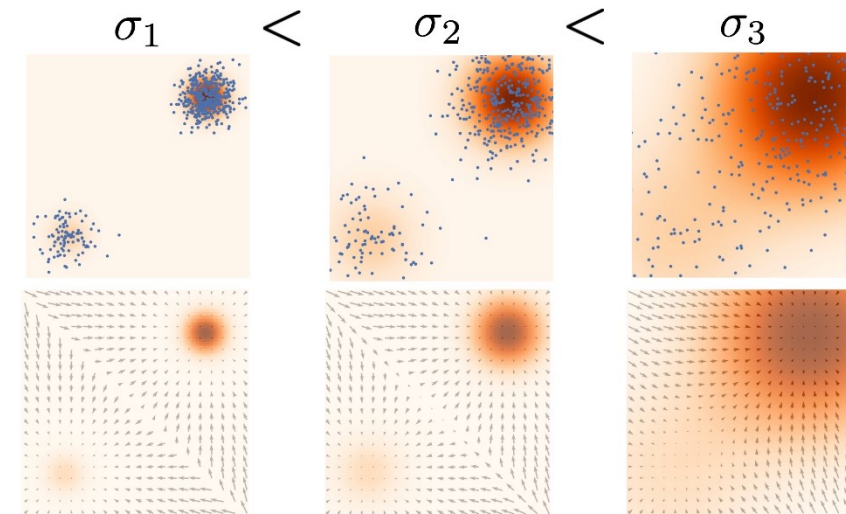
$$p_{\sigma_i}(\mathbf{x}) = \int p(\mathbf{y}) \mathcal{N}(\mathbf{x}; \mathbf{y}, \sigma_i^2 I) d\mathbf{y}.$$

$$\mathbf{s}_{\theta}(\mathbf{x}, i) \approx \nabla_{\mathbf{x}} \log p_{\sigma_i}(\mathbf{x}) \quad \text{Noise Conditional Score-Based Model}$$

$$\sum_{i=1}^L \lambda(i) \mathbb{E}_{p_{\sigma_i}(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p_{\sigma_i}(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x}, i)\|_2^2],$$

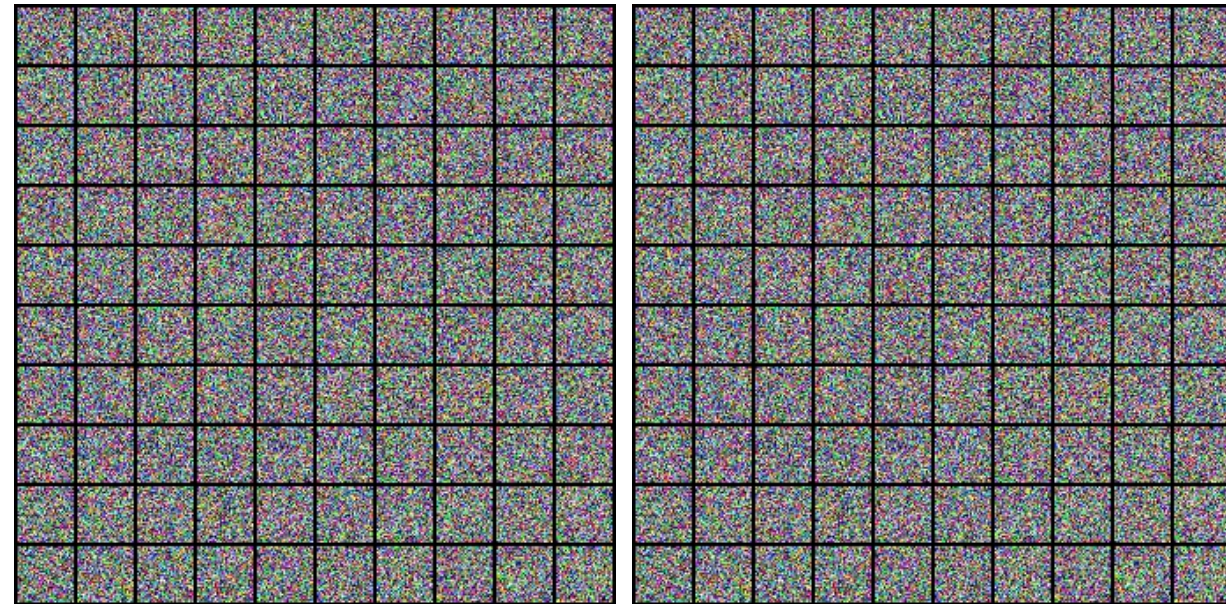
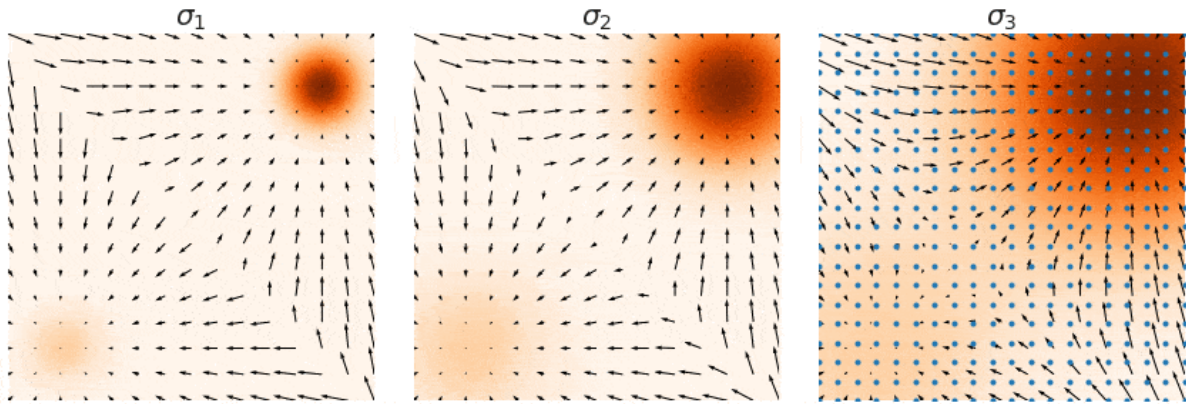
$$\lambda(i) = \sigma_i^2$$

Parameterize the score-based model $\mathbf{s}_{\theta}(\mathbf{x}, i)$ with U-Net skip connections



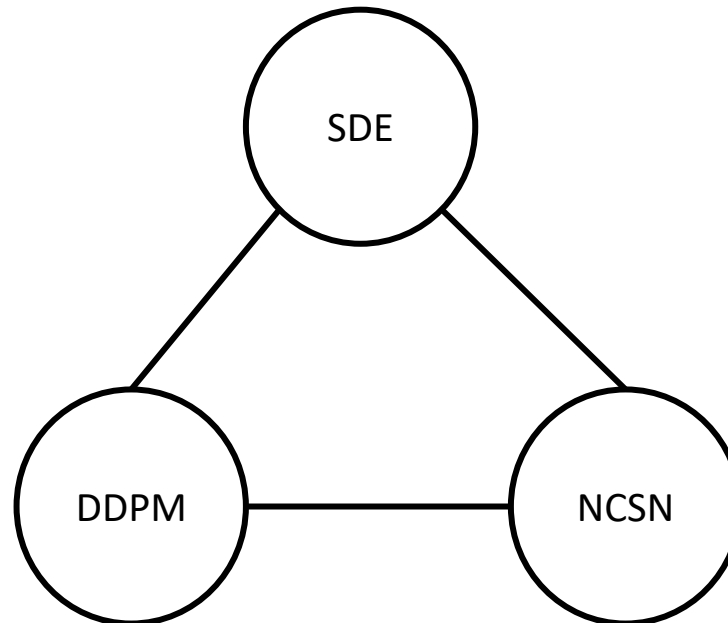
Score-based generative modeling with multiple noise perturbations

- After training our noise-conditional score-based model $\mathbf{s}_\theta(\mathbf{x}, i)$, we can produce samples from it by running Langevin dynamics for $i = L, L - 1, \dots, 1$ in sequence.
- This method is called annealed Langevin dynamics, since the noise scale σ_i decreases (anneals) gradually over time.



Diffusion Models: Unification with SDE

- We've just seen two different approaches for diffusion models:
 1. DDPM: ELBO objective
 2. NCSN (Noise Conditional Score Matching): score-matching objective
- These two show commonalities in both objective & sampling procedures
 - Add noise to original data to destruct its structure
 - Sampling: starts from a prior, steadily decreases noise level
- Can we **unify** them?



Published as a conference paper at ICLR 2021

SCORE-BASED GENERATIVE MODELING THROUGH STOCHASTIC DIFFERENTIAL EQUATIONS

Yang Song*
Stanford University
yangsong@cs.stanford.edu

Jascha Sohl-Dickstein
Google Brain
jaschasd@google.com

Diederik P. Kingma
Google Brain
durk@google.com

Abhishek Kumar
Google Brain
abhishk@google.com

Stefano Ermon
Stanford University
ermon@cs.stanford.edu

Ben Poole
Google Brain
pooleb@google.com

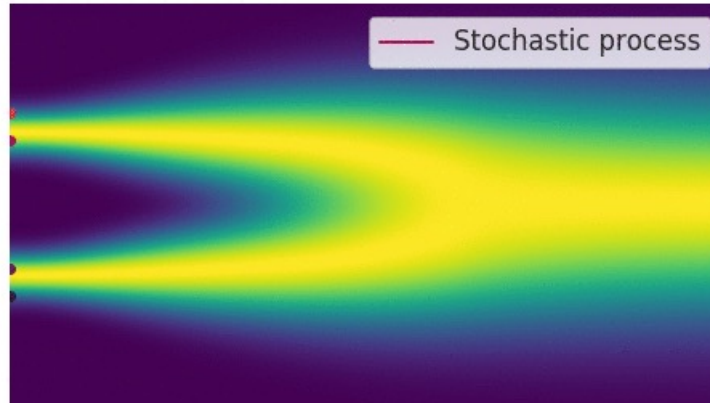
ABSTRACT

Creating noise from data is easy; creating data from noise is generative modeling. We present a stochastic differential equation (SDE) that smoothly transforms a complex data distribution to a known prior distribution by slowly injecting noise, and a corresponding reverse-time SDE that transforms the prior distribution back into the data distribution by slowly removing the noise. Crucially, the reverse-time SDE depends only on the time-dependent gradient field (a.k.a., score) of the perturbed data distribution. By leveraging advances in score-based generative modeling, we can accurately estimate these scores with neural networks, and use numerical SDE solvers to generate samples. We show that this framework encapsulates previous approaches in score-based generative modeling and diffusion probabilistic modeling, allowing for new sampling procedures and new modeling capabilities. In particular, we introduce a predictor-corrector framework to correct errors in the evolution of the discretized reverse-time SDE. We also derive an equivalent neural ODE that samples from the same distribution as the SDE, but additionally enables exact likelihood computation, and improved sampling efficiency. In addition, we provide a new way to solve inverse problems with score-based models, as demonstrated with experiments on class-conditional generation, image inpainting, and colorization. Combined with multiple architectural improvements, we achieve record-breaking performance for unconditional image generation on CIFAR-10 with an Inception score of 9.89 and FID of 2.20, a competitive likelihood of 2.99 bits/dim, and demonstrate high fidelity generation of 1024×1024 images for the first time from a score-based generative model.

Score-based generative modeling with stochastic differential equations (SDEs)

Perturbing data with an SDE

- When **the number of noise scales approaches infinity**, we essentially perturb the data distribution with continuously growing levels of noise.
- In this case, **the noise perturbation procedure is a continuous-time stochastic process** as demonstrated below



$\{\mathbf{x}(t)\}_{t \in [0, T]}$: The solution
 $p_t(\mathbf{x})$: The marginal pdf of $\mathbf{x}(t)$
 $p_0(\mathbf{x})$: The data distribution

- How can we represent a stochastic process in a concise way?

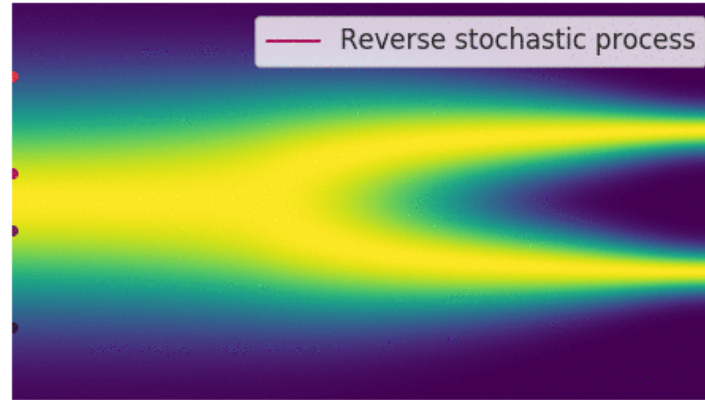
$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$$

- ✓ $\mathbf{f}(\cdot, t): \mathbb{R}^d \rightarrow \mathbb{R}^d$ is a vector-valued function called the drift coefficient
- ✓ $g(t) \in \mathbb{R}$ is a real-valued function called the diffusion coefficient,
- ✓ $\mathbf{w}$ denotes a standard Brownian motion, and $d\mathbf{w}$ can be viewed as infinitesimal white noise.

Score-based generative modeling with stochastic differential equations (SDEs)

Reversing the SDE for sample generation

- With a finite number of noise scales, we can generate samples by reversing the perturbation process with annealed Langevin dynamics
- For infinite noise scales, we can analogously reverse the perturbation process by using the reverse SDE.



$\{\mathbf{x}(t)\}_{t \in [0, T]}$: The solution
 $p_t(\mathbf{x})$: The marginal pdf of $\mathbf{x}(t)$
 $p_0(\mathbf{x})$: The data distribution

- Any SDE has a corresponding reverse SDE, whose closed form is given by

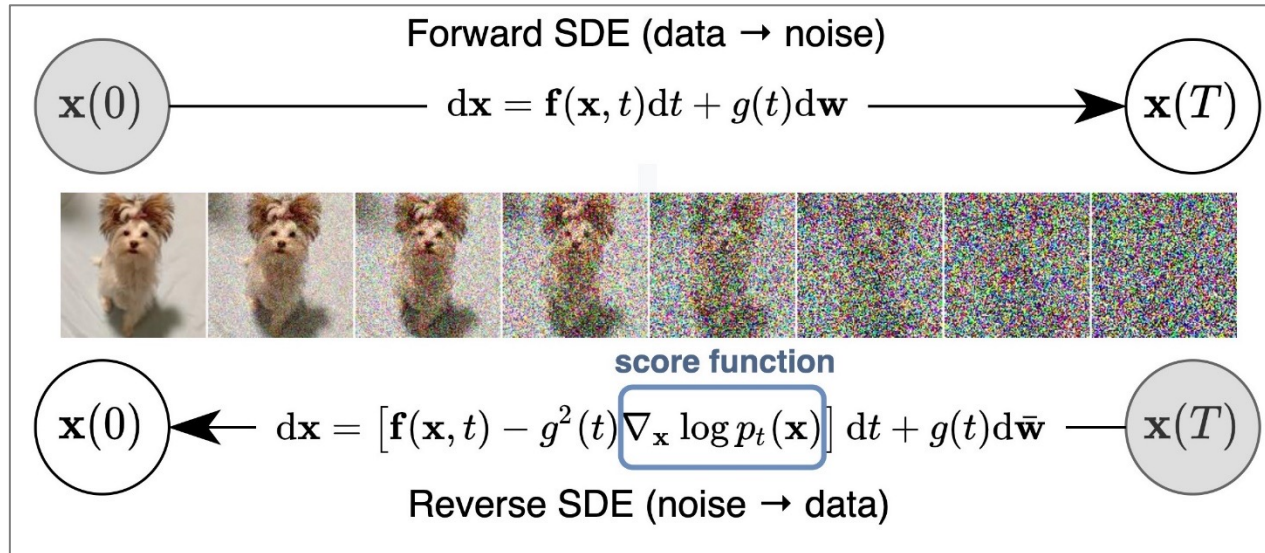
$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t)\nabla_{\mathbf{x}}\log p_t(\mathbf{x})]dt + g(t)d\mathbf{w}$$

- ✓ Here dt represents a negative infinitesimal time step
- ✓ The SDE needs to be solved backwards in time (from $t = T$ to $t = 0$).

Score-based generative modeling with stochastic differential equations (SDEs)

Estimating the reverse SDE with score-based models and score matching

- Solving the reverse SDE requires us to know the terminal distribution $p_T(x)$, and the score function $\nabla_x \log p_t(x)$.



- In order to estimate $\nabla_x \log p_t(\mathbf{x})$, we train a **Time-Dependent Score-Based Model** $\mathbf{s}_\theta(\mathbf{x}, t) \approx \nabla_x \log p_t(\mathbf{x})$
 - ✓ This is analogous to the noise-conditional score-based model $\mathbf{s}_\theta(\mathbf{x}, i) \approx \nabla_x \log p_{\sigma_i}(\mathbf{x})$
- Training objective for $\mathbf{s}_\theta(\mathbf{x}, t)$ is a continuous weighted combination of Fisher divergences, given by

$$\mathbb{E}_{t \in \mathcal{U}(0, T)} \mathbb{E}_{p_t(\mathbf{x})} [\lambda(t) \|\nabla_x \log p_t(\mathbf{x}) - \mathbf{s}_\theta(\mathbf{x}, t)\|_2^2],$$

Diffusion Models: Unification with SDE

KEY IDEA: We have SDEs that correspond to NCSN & DDPMs

- SDE-version of NCSN: Variance exploding (VE) SDE

$$dx = \sqrt{\frac{d[\sigma^2(t)]}{dt}} dw$$

- SDE-version of DDPM: Variance preserving (VP) SDE

$$dx = -\frac{1}{2}\beta(t)xdt + \sqrt{\beta(t)}dw$$

- New type of SDE-suggested

$$dx = -\frac{1}{2}\beta(t)xdt + \sqrt{\beta(t) \left(1 - e^{-2 \int_0^t \beta(s)ds}\right)} dw$$

Summary

	Objective	Sampling
VAE → DDPM	$E_t E_{x_0} E_{q(x_t x_0)} [D_{KL}(q(x_{t-1} x_t, x_0), p(x_{t-1} x_t))]$	Repeatedly running trained generator $p_\theta(x_{t-1} x_t)$
EBM → NCSN	$L(\theta; \{\sigma_i\}_{i=1}^N) = \frac{1}{2L} \sum_{i=1}^L \lambda(\sigma_i) E_{p_{data}(x)} E_{\tilde{x} \sim N(x, \sigma^2 I)} \left[\left\ s_\theta(\tilde{x}, \sigma) + \frac{\tilde{x} - x}{\sigma^2} \right\ ^2 \right]$	Annealed Langevin dynamics
SDE	$E_t \{ \lambda(t) E_{x(0)} E_{x(t) x(0)} \left[\left\ s_\theta(x(t), t) - \nabla_{x(t)} \log p_{0t}(x(t) x(0)) \right\ ^2 \right] \}$	Reverse-time SDE

Controllable generation for inverse problem solving (DDPM)

4.1 Conditional Reverse Noising Process

We start with a diffusion model with an unconditional reverse noising process $p_\theta(x_t|x_{t+1})$. To condition this on a label y , it suffices to sample each transition² according to

$$p_{\theta,\phi}(x_t|x_{t+1}, y) = Z p_\theta(x_t|x_{t+1}) p_\phi(y|x_t) \quad (2)$$

Algorithm 1 Classifier guided diffusion sampling, given a diffusion model $(\mu_\theta(x_t), \Sigma_\theta(x_t))$, classifier $p_\phi(y|x_t)$, and gradient scale s .

Input: class label y , gradient scale s

$x_T \leftarrow$ sample from $\mathcal{N}(0, \mathbf{I})$

for all t from T to 1 **do**

$\mu, \Sigma \leftarrow \mu_\theta(x_t), \Sigma_\theta(x_t)$

$x_{t-1} \leftarrow$ sample from $\mathcal{N}(\mu + s\Sigma \nabla_{x_t} \log p_\phi(y|x_t), \Sigma)$

end for

return x_0



Figure 3: Samples from an unconditional diffusion model with classifier guidance to condition on the class "Pembroke Welsh corgi". Using classifier scale 1.0 (left; FID: 33.0) does not produce convincing samples in this class, whereas classifier scale 10.0 (right; FID: 12.0) produces much more class-consistent images.

- A classifier $p_\phi(y|x_t)$ that guides the denoising model towards the desired subset of images with the attribute y is trained in parallel with the denoiser.

Dhariwal and Nichol, "Diffusion Models Beat GANs on Image Synthesis",

Controllable generation for inverse problem solving (SMD)

- Score-based generative models are particularly suitable for solving inverse problems. At its core, inverse problems are same as Bayesian inference problems

$$p(x|y) = \frac{p(y|x)p(x)}{\int p(x)p(y|x)dx}$$

- This expression can be greatly simplified by taking gradients with respect to x on both sides, leading to the following Bayes' rule for score functions:

$$\nabla_x \log p(x|y) = \nabla_x \log p(x) + \nabla_x \log p(y|x)$$

- Through score matching, we can train a model to estimate the score function of the unconditional data distribution, i.e., $s_\theta(x) \approx \nabla_x \log p(x)$.

→ This will allow us to easily compute the posterior score function $\nabla_x \log p(x|y)$ from the known forward process $p(y|x)$, and sample from it with Langevin-type sampling.

GEODIFF: A GEOMETRIC DIFFUSION MODEL FOR MOLECULAR CONFORMATION GENERATION

Minkai Xu^{1,2}, Lantao Yu³, Yang Song³, Chence Shi^{1,2}, Stefano Ermon^{3*}, Jian Tang^{1,4,5*}

¹Mila - Québec AI Institute, Canada ²Université de Montréal, Canada

³Stanford University, USA ⁴HEC Montréal, Canada ⁵CIFAR AI Research Chair

{minkai.xu, chence.shi}@umontreal.ca

{lantaoyu, yangsong, ermon}@cs.stanford.edu

jian.tang@hec.ca

ABSTRACT

Predicting molecular conformations from molecular graphs is a fundamental problem in cheminformatics and drug discovery. Recently, significant progress has been achieved with machine learning approaches, especially with deep generative models. Inspired by the diffusion process in classical non-equilibrium thermodynamics where heated particles will diffuse from original states to a noise distribution, in this paper, we propose a novel generative model named GEODIFF for molecular conformation prediction. GEODIFF treats each atom as a particle and learns to directly reverse the diffusion process (*i.e.*, transforming from a noise distribution to stable conformations) as a Markov chain. Modeling such a generation process is however very challenging as the likelihood of conformations should be roto-translational invariant. We theoretically show that Markov chains evolving with *equivariant* Markov kernels can induce an *invariant* distribution by design, and further propose building blocks for the Markov kernels to preserve the desirable equivariance property. The whole framework can be efficiently trained in an end-to-end fashion by optimizing a weighted variational lower bound to the (conditional) likelihood. Experiments on multiple benchmarks show that GEODIFF is superior or comparable to existing state-of-the-art approaches, especially on large molecules.¹

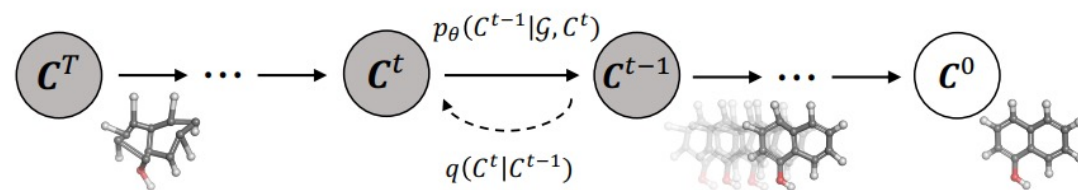


Figure 1: Illustration of the diffusion and reverse process of GEODIFF. For diffusion process, noise from fixed posterior distributions $q(C^t|C^{t-1})$ is gradually added until the conformation is destroyed. Symmetrically, for generative process, an initial state C^T is sampled from standard Gaussian distribution, and the conformation is progressively refined via the Markov kernels $p_\theta(C^{t-1}|G, C^t)$.

Diffusion process

$$q(C^{1:T}|C^0) = \prod_{t=1}^T q(C^t|C^{t-1}), \quad q(C^t|C^{t-1}) = \mathcal{N}(C^t; \sqrt{1 - \beta_t}C^{t-1}, \beta_t I).$$

Denoising process

$$p_\theta(C^{0:T-1}|G, C^T) = \prod_{t=1}^T p_\theta(C^{t-1}|G, C^t), \quad p_\theta(C^{t-1}|G, C^t) = \mathcal{N}(C^{t-1}; \mu_\theta(G, C^t, t), \sigma_t^2 I).$$

specified molecular graphs G

Published as a conference paper at ICLR 2023

DIFFUSION PROBABILISTIC MODELING OF PROTEIN BACKBONES IN 3D FOR THE MOTIF-SCAFFOLDING PROBLEM

Brian L. Trippe^{*†}
btrippe@mit.edu

Jason Yim^{*†}
jyim@mit.edu

Doug Tischer[‡]
dtischer@uw.edu

David Baker[‡]
dabaker@uw.edu

Tamara Broderick[†]
tbroderick@mit.edu

Regina Barzilay[†]
regina@csail.mit.edu

Tommi Jaakkola[†]
tommi@csail.mit.edu

ABSTRACT

Construction of a *scaffold* structure that supports a desired *motif*, conferring protein function, shows promise for the design of vaccines and enzymes. But a general solution to this motif-scaffolding problem remains open. Current machine-learning techniques for scaffold design are either limited to unrealistically small scaffolds (up to length 20) or struggle to produce multiple diverse scaffolds. We propose to learn a distribution over diverse and longer protein backbone structures via an E(3)-equivariant graph neural network. We develop **SMCDiff** to efficiently sample scaffolds from this distribution conditioned on a given motif; our algorithm is the first to theoretically guarantee conditional samples from a diffusion model in the large-compute limit. We evaluate our designed backbones by how well they align with AlphaFold2-predicted structures. We show that our method can (1) sample scaffolds up to 80 residues and (2) achieve structurally diverse scaffolds for a fixed motif.¹

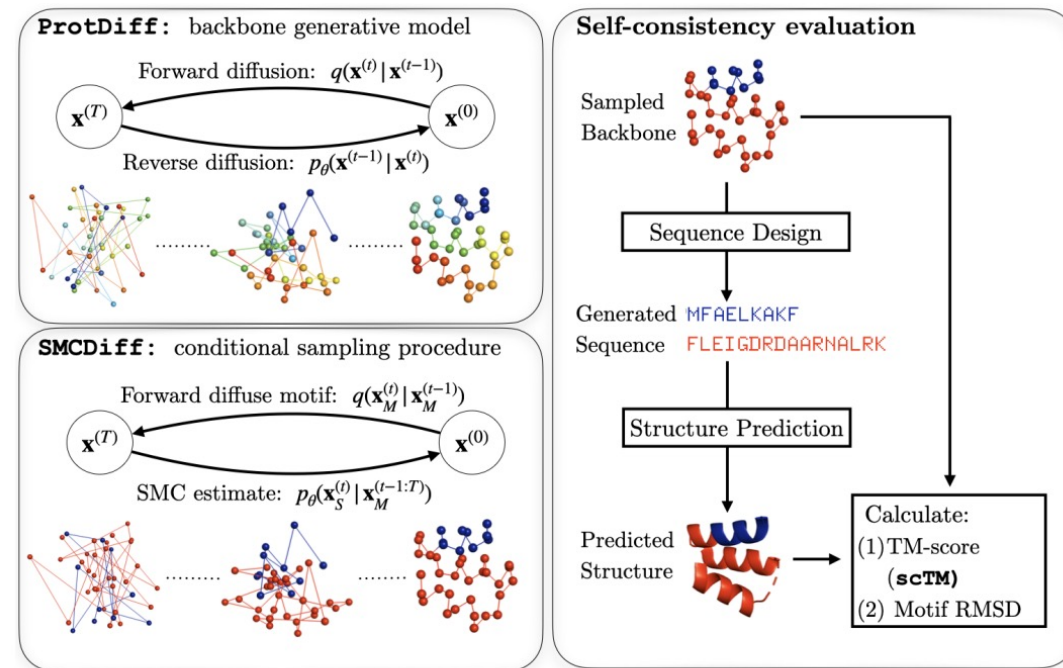


Figure 1: Overview of the conditional generative modeling approach to the motif-scaffolding problem. We train our new protein backbone diffusion model, **ProtDiff**, to generate realistic protein backbone structures. Next, we run **SMCDiff**, our conditional sampling algorithm, with **ProtDiff** to generate scaffolds (colored in red) conditioned on the motif (colored in blue). For self-consistency evaluation, we use a pretrained fixed-backbone sequence-design model (**ProteinMPNN** (Dauparas et al., 2022)) to generate the scaffold sequence from a sampled backbone. We then input the sequence to a structure prediction model, in our case **AlphaFold2** (AF2) (Jumper et al., 2021), to generate the full protein structure from the generated sequence. We compare the backbone of the predicted structure with the original backbone structure using TM-score (Xu & Zhang, 2010) and root-mean-square-distance (RMSD) for the motif.

Diffusion Models Beat GANs on Topology Optimization

François Mazé¹, Faez Ahmed¹

¹Massachusetts Institute of Technology
francois.maze@etu.minesparis.psl.eu, {fmaze, faez}@mit.edu

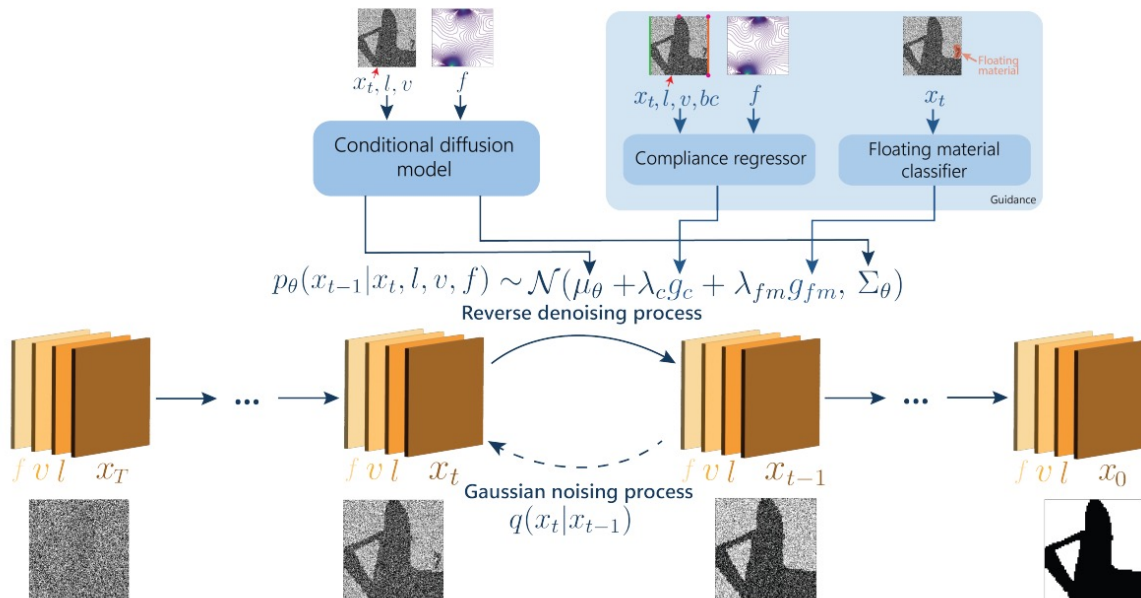


Figure 2: TopoDiff: Proposed constrained guided conditional diffusion model architecture for TO. $(x_t)_{t=0, \dots, T}$ is the gradually denoised topology; g_c and g_{fm} are the guidance gradients; l is the load applied, represented with an arrow on the topology; v is the volume fraction; f are the physical fields, and bc are the boundary conditions, represented with color lines and dots.

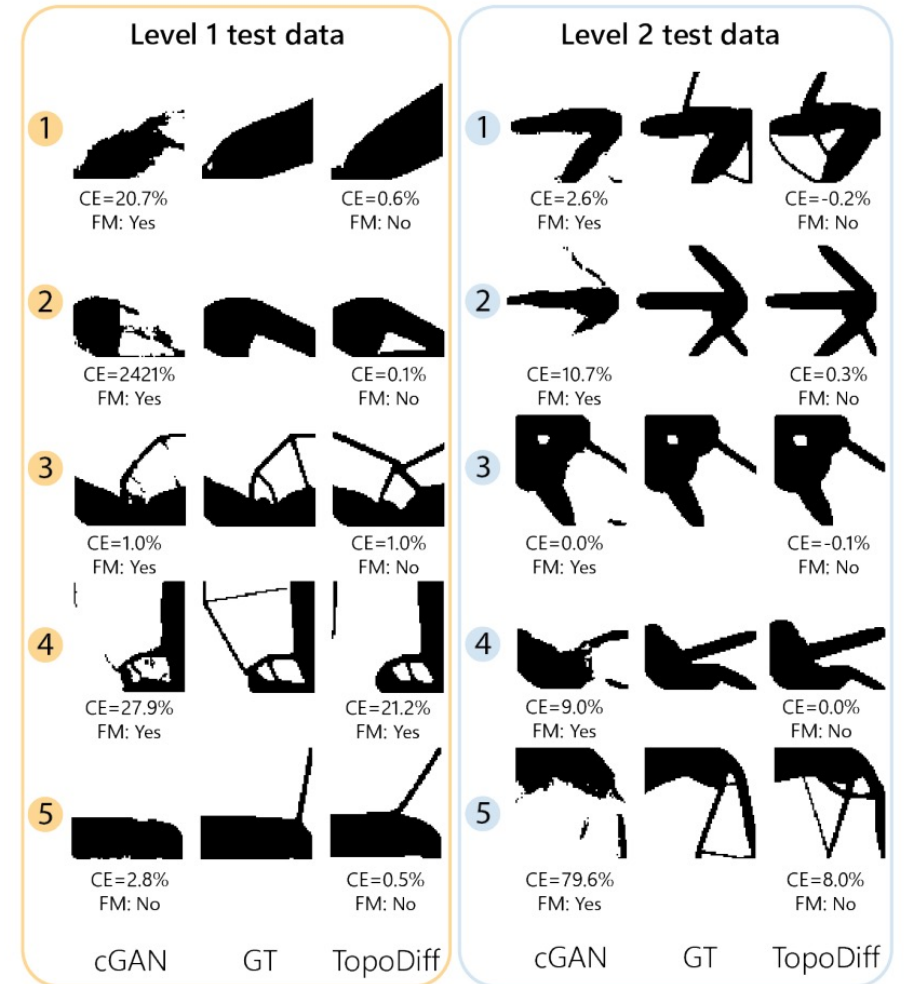


Figure 3: Comparison of generated structures on randomly selected samples from both test datasets. GT stands for ground truth, CE is the compliance error relative to the GT, and FM indicates the presence or not of floating material.

Papers fall into Inverse approaches

- Conditioning by Adaptive Sampling (CbAS, ICML 2019)
- Model Inversion Networks (MINs, NIPS 2020)

More Recent Papers

- Diffusion models for Black-box Optimization (DDOM, ICML 2023)
- Generative Pretraining for Black-box Optimization (BONET, ICML 2023)
- Bootstrapped Training of Score-conditioned Generator for Offline Design of Biological Sequences (BootGen, NIPS 2023)

Questions?

?

References

- [1] Design-Bench: Benchmarks for Data-Driven Offline Model-Based Optimization (<https://arxiv.org/abs/2202.08450>)
- [2] What are Diffusion Models? – Lil'Log (<https://lilianweng.github.io/posts/2021-07-11-diffusion-models>)
- [3] Stanford CS231N Lecture 13. Generative Models (http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture13.pdf)

Supplementary

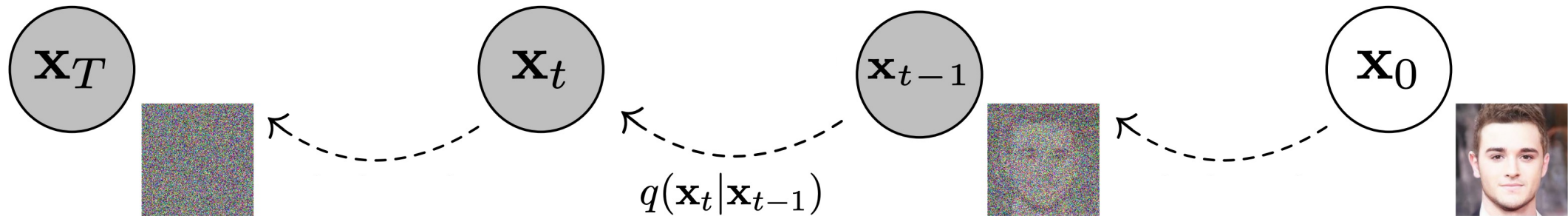
The starting point

Deep Unsupervised Learning using Nonequilibrium Thermodynamics, Jascha Sohl-Dickstein et al., ICML 2015



- Proposed the theoretical foundations of denoising diffusion models.

Forward and reverse processes



- We gradually corrupt the data x_0 and create noisy data $x_{1:T}$ by injecting noise on each step t using
$$q(x_t | x_{t-1}) \triangleq T(x_t | x_{t-1}; \beta_t)$$
- As an example, when we employ Gaussian Kernel, then $q(x_t | x_{t-1})$ can be designed as $\mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$.
- We refer to the joint distribution of $q(x_{0:T}) = q(x_0) \prod_{t=1}^T q(x_t | x_{t-1})$ as forward process. – a stochastic process.
- The limit of the forward process, i.e., x_T with sufficiently large T , is designed to converge to a simple and well-behaved distribution such as $\mathcal{N}(x_T; 0, I)$.

Figure source: [Denoising Diffusion Probabilistic Models](#), Ho et al 2020.

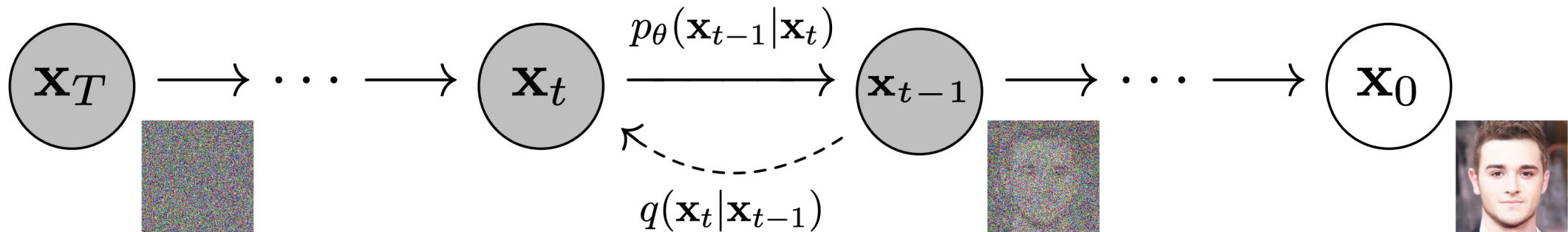
Forward and reverse processes

- There are some nice properties when we set $q(x_t|x_{t-1})$ as $\mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$.
- We can sample x_t conditioned on x_0 directly w/o relying on the iterative procedure.
- Let $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$, and $\epsilon \sim \mathcal{N}(0, I)$ then

$$\begin{aligned}x_t &= \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon \quad (*) \\&= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_t\alpha_{t-1}}\epsilon \\&= \dots \\&= \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \\q(x_t|x_0) &= \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t}x_0, \sqrt{1 - \bar{\alpha}_t}I\right)\end{aligned}$$

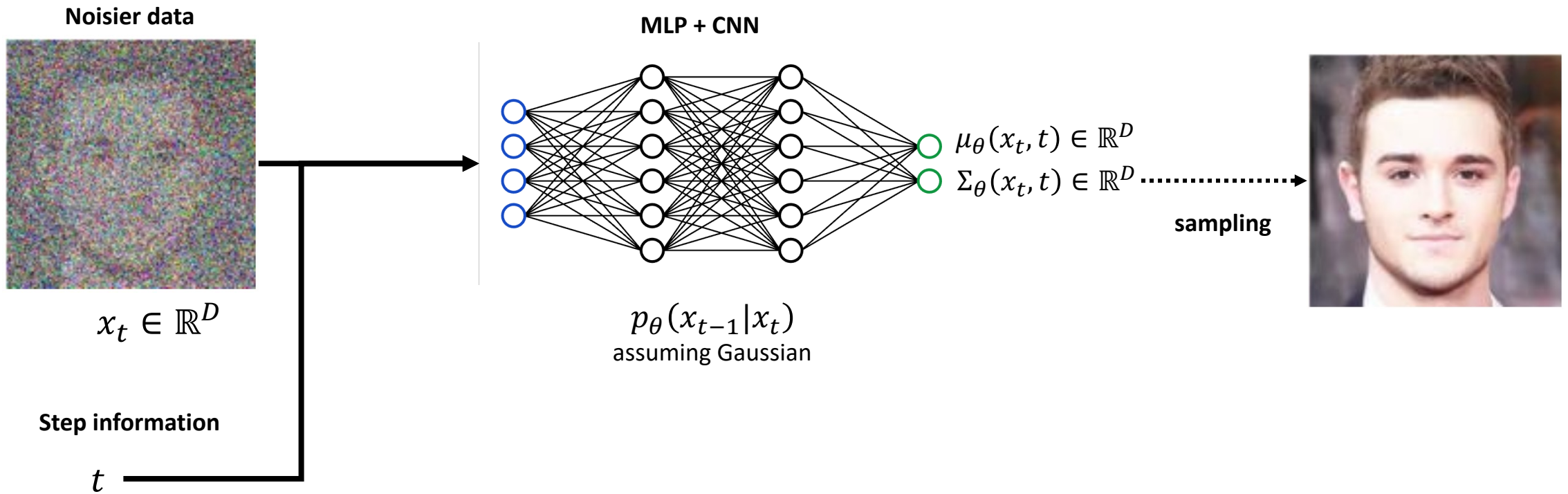
- $(*)$ Sum of the two independent Gaussians with different variance, $\mathcal{N}(0, \sigma_1^2 I)$ and $\mathcal{N}(0, \sigma_2^2 I)$ follows $\mathcal{N}(0, (\sigma_1^2 + \sigma_2^2)I)$

Forward and Reverse processes



- Want to learn how to iteratively denoise x_T so that we can generate $x_0 \sim q(x_0)$ where $q(x_0)$ is the data distribution.
- At each step t , we denoise the noisier data x_t to less noisy data x_{t-1} by using $p_\theta(x_{t-1}|x_t)$.
- When the forward Kernel is Gaussian/Binomial, $p_\theta(x_{t-1}|x_t)$ has the identical functional form. [Feller 1949]
- We refer to the joint distribution $p_\theta(x_{0:T}) \triangleq p(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t)$ as reverse (or denoise) process.

Network architecture



The Learning Objective

$$\begin{aligned}\mathcal{L}_{CE}(\theta) &= -\mathbb{E}_{q(x_0)} \log p_\theta(x_0) \\ &= -\mathbb{E}_{q(x_0)} \log \left(\int p_\theta(x_{0:T}) dx_{1:T} \right) \\ &= -\mathbb{E}_{q(x_0)} \log \left(\int q(x_{1:T}|x_0) \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} dx_{1:T} \right) \\ &= -\mathbb{E}_{q(x_0)} \log \left(\mathbb{E}_{q(x_{1:T}|x_0)} \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right) \\ &\leq \mathbb{E}_{q(x_{0:T})} \log \left(\frac{q(x_{1:T}|x_0)}{p_\theta(x_{0:T})} \right) = -\mathcal{L}_{ELBO}(\theta)\end{aligned}$$

Loss function

The Learning Objective

$$\begin{aligned}
 \mathcal{L}(\theta) &= \mathbb{E}_{q(x_{0:T})} \log \left(\frac{q(x_{1:T}|x_0)}{p_\theta(x_{0:T})} \right) \\
 &= \mathbb{E}_{q(x_{0:T})} \log \left(\frac{\prod_{t=1}^T q(x_t|x_{t-1})}{p(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t)} \right) \\
 &= \mathbb{E}_{q(x_{0:T})} \left[-\log p(x_T) + \sum_{t=2}^T \log \frac{q(x_t|x_{t-1})}{p_\theta(x_{t-1}|x_t)} + \log \frac{q(x_1|x_0)}{p_\theta(x_0|x_1)} \right] \\
 &= \mathbb{E}_{q(x_{0:T})} \left[-\log p(x_T) + \sum_{t=2}^T \log \frac{q(x_t|x_{t-1}, x_0)}{p_\theta(x_{t-1}|x_t)} + \log \frac{q(x_1|x_0)}{p_\theta(x_0|x_1)} \right] \\
 &= \mathbb{E}_{q(x_{0:T})} \left[-\log p(x_T) + \sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_\theta(x_{t-1}|x_t)} \frac{q(x_t|x_0)}{q(x_{t-1}|x_0)} + \log \frac{q(x_1|x_0)}{p_\theta(x_0|x_1)} \right] \\
 &= \mathbb{E}_{q(x_{0:T})} \left[-\log p(x_T) + \sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_\theta(x_{t-1}|x_t)} + \log \frac{q(x_1|x_0)}{p_\theta(x_0|x_1)} \right] \quad \because q(x_t|x_{t-1}) = \frac{q(x_{t-1}|x_t)q(x_t)}{q(x_{t-1})} \\
 &= \mathbb{E}_{q(x_{0:T})} \left[\log \frac{q(x_T|x_0)}{p(x_T)} + \sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_\theta(x_{t-1}|x_t)} - \log p_\theta(x_0|x_1) \right] \\
 &= \mathbb{E}_{q(x_{0:T})} \left[\underbrace{D_{\text{KL}}(q(x_T|x_0) || p_\theta(x_T))}_{L_T} + \underbrace{\sum_{t=2}^T D_{\text{KL}}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t))}_{L_{t-1}} - \underbrace{\log p_\theta(x_0|x_1)}_{L_0} \right]
 \end{aligned}$$

How to compute L_t efficiently?

- L_T : $D_{\text{KL}}(q(x_T|x_0)||p_\theta(x_T))$ can be computed analytically.
- L_0 : $\log p_\theta(x_0|x_1)$ can be estimated via sampling.
- L_{t-1} : $D_{\text{KL}}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))$
 - $p_\theta(x_{t-1}|x_t)$ is our model. i.e., Gaussian or Binomial
 - What about $q(x_{t-1}|x_t, x_0)$?

In fact, $q(x_{t-1}|x_t, x_0)$ is Gaussian.

$$\begin{aligned} q(x_{t-1}|x_t, x_0) &= q(x_t|x_{t-1}, x_0) \frac{q(x_{t-1}|x_0)}{q(x_t|x_0)} \\ &\propto \exp\left(-\frac{1}{2}\left(\frac{(x_t - \sqrt{\alpha_t}x_{t-1})^2}{\beta_t} + \frac{(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}}x_0)^2}{1 - \bar{\alpha}_{t-1}} - \frac{(x_t - \sqrt{\bar{\alpha}_t}x_0)^2}{1 - \bar{\alpha}_t}\right)\right) \\ &= \exp\left(\left(-\frac{1}{2}\left(\frac{x_t^2 - 2\sqrt{\alpha_t}x_tx_{t-1} + \alpha_tx_{t-1}^2}{\beta_t} + \frac{x_{t-1}^2 - 2\sqrt{\bar{\alpha}_{t-1}}x_{t-1}x_0 + \bar{\alpha}_{t-1}x_0^2}{1 - \bar{\alpha}_{t-1}} - \frac{(x_t - \sqrt{\bar{\alpha}_t}x_0)^2}{1 - \bar{\alpha}_t}\right)\right)\right) \\ &= \exp\left(\left(-\frac{1}{2}\left(\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}\right)x_{t-1}^2 - \left(\frac{2\sqrt{\alpha_t}}{\beta_t}x_t + \frac{2\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}}x_0\right)x_{t-1} + C(x_t, x_0)\right)\right)\right) \end{aligned}$$

In fact, $q(x_{t-1}|x_t, x_0)$ is Gaussian.

DOUBLE CHECK THE DERIVATIONS!

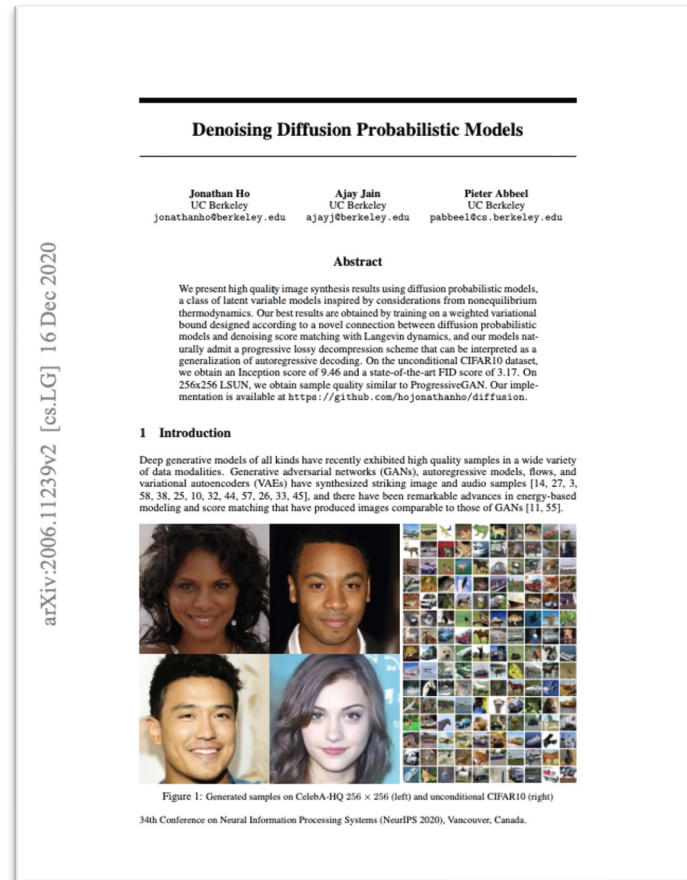
- $\tilde{\beta}_t = 1/\left(\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}}\right) = 1/\left(\frac{\alpha_t(1-\bar{\alpha}_{t-1})}{\beta_t(1-\bar{\alpha}_{t-1})} + \frac{\beta_t}{\beta_t(1-\bar{\alpha}_{t-1})}\right) = 1/\left(\frac{\alpha_t - \bar{\alpha}_t + \beta_t}{\beta_t(1-\bar{\alpha}_{t-1})}\right) = 1/\left(\frac{(1-\beta_t) - \bar{\alpha}_t + \beta_t}{\beta_t(1-\bar{\alpha}_{t-1})}\right) = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \beta_t$
- $\tilde{\mu}_t(x_t, x_0) = \left(\frac{\sqrt{\alpha_t}}{\beta_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}} x_0\right) / \left(\frac{\alpha_t}{\beta_t} + \frac{1}{1-\bar{\alpha}_{t-1}}\right) = \left(\frac{\sqrt{\alpha_t}}{\beta_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}} x_0\right) \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \beta_t$
$$= \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}{1-\bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t} x_0$$
$$= \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}{1-\bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t} x_0$$
- $q(x_{t-1}|x_t, x_0) = \mathcal{N}(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$
- Another parameterization of $\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}{1-\bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t} x_0 = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}{1-\bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t} \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \sqrt{1-\bar{\alpha}_t} \epsilon_t\right)$
$$= \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_t\right)$$

How to compute L_t efficiently?

- L_T : $D_{\text{KL}}(q(x_T|x_0)||p_\theta(x_T))$ can be computed analytically.
- L_0 : $\log p_\theta(x_0|x_1)$ can be estimated via sampling.
- L_{t-1} : $D_{\text{KL}}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))$
 - $p_\theta(x_{t-1}|x_t)$ is our model. i.e., Gaussian or Binomial
- What about $q(x_{t-1}|x_t, x_0)$?
 - $q(x_{t-1}|x_t, x_0)$ and $p_\theta(x_{t-1}|x_t)$ are Gaussian $\rightarrow$ We can compute the KL terms analytically.
- That is, we can compute $\mathcal{L}(\theta)$ tractably.

Make things more practical

Denoising Diffusion Probabilistic Models Jonathan Ho et al., NeurIPS 2020



- Proposed clever algorithmic choices.

Algorithmic choices

1. Schedule the forward process variances β_t as constants rather than learning them.
2. Set $\Sigma_\theta(x_t, t)$ as $\sigma_t I$ and experimentally confirmed that $\sigma_t = \beta_t$ or $\frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \beta_t$ work well.
3. Learn $\mu_\theta(x_t, t)$ only.
4. When learning $\mu_\theta(x_t, t)$, propose to learn error terms only.

In-depth Inspection of L_{t-1}

$$\begin{aligned} L_{t-1} &= \mathbb{E}_q \left[D_{\text{KL}} \left(q(x_{t-1} | x_t, x_0) || p_\theta(x_{t-1} | x_t) \right) \right] \\ &= \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \left\| \tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t) \right\|^2 \right] + C \end{aligned}$$

KL-divergence bet. two Gaussians

- We can simplify L_{t-1} using:

$$x_t(x_0, \epsilon) = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \text{ for } \epsilon \sim \mathcal{N}(0, I) \text{ and } \tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0$$

$$\begin{aligned} L_{t-1} - C &= \mathbb{E}_{x_0, \epsilon} \left[\frac{1}{2\sigma_t^2} \left\| \tilde{\mu}_t \left(x_t, \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t(x_0, \epsilon) - \sqrt{1 - \bar{\alpha}_t} \epsilon \right) \right) - \mu_\theta(x_t, t) \right\|^2 \right] \\ &= \mathbb{E}_{x_0, \epsilon} \left[\frac{1}{2\sigma_t^2} \left\| \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t(x_0, \epsilon) - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right) - \mu_\theta(x_t, t) \right\|^2 \right] \end{aligned}$$

The detailed derivation can be found from the supplementary slide.

In-depth Inspection of L_{t-1}

$$\begin{aligned} L_{t-1} &= \mathbb{E}_q \left[D_{\text{KL}}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t)) \right] \\ &= \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2 \right] + C \end{aligned}$$

KL-divergence bet. two Gaussians

- We can simplify L_{t-1} using:

$$x_t(x_0, \epsilon) = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \text{ for } \epsilon \sim \mathcal{N}(0, I) \text{ and } \tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)}{1 - \bar{\alpha}_t}x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0$$

$$\begin{aligned} L_{t-1} - C &= \mathbb{E}_{x_0, \epsilon} \left[\frac{1}{2\sigma_t^2} \left\| \tilde{\mu}_t \left(x_t, \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t(x_0, \epsilon) - \sqrt{1 - \bar{\alpha}_t}\epsilon \right) \right) - \mu_\theta(x_t, t) \right\|^2 \right] \\ &= \mathbb{E}_{x_0, \epsilon} \left[\frac{1}{2\sigma_t^2} \left\| \frac{1}{\sqrt{\alpha_t}} \left(x_t(x_0, \epsilon) - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon \right) - \mu_\theta(x_t, t) \right\|^2 \right] \end{aligned}$$

- The simplified L_{t-1} suggests that $\mu_\theta(x_t, t)$ must predict $\frac{1}{\sqrt{\alpha_t}} \left(x_t(x_0, \epsilon) - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon \right)$ given x_0 and t .

The detailed derivation can be found from the supplementary slide.

Simplify the learning objective via reparametrization

- Our prediction target $\tilde{\mu}_t \left(x_t, \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t(x_0, \epsilon) - \sqrt{1 - \bar{\alpha}_t} \epsilon \right) \right) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t(x_0, \epsilon) - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right)$ and $x_t(x_0, \epsilon)$ is given at the training and inference phase.
- Hence why don't we learn only error term by setting $\mu_\theta \stackrel{\text{def}}{=} \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t(x_0, \epsilon) - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$?
- To sample $x_{t-1} \sim p_\theta(x_{t-1}|x_t)$, we can compute $x_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z$, where $z \sim \mathcal{N}(0, I)$.
- With the reparameterization, L_{t-1} becomes:

$$L_{t-1} = \mathbb{E}_{x_0, \epsilon} \left[\frac{\beta_t^2}{2\sigma_t^2 \bar{\alpha}_t (1 - \bar{\alpha}_t)} \left\| \epsilon - \epsilon_\theta \left(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2 \right]$$

Further Simplification on Loss

- The authors observe that simple “reweighted” version of L_{t-1} works even better in practice.

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \left[\left\| \epsilon - \epsilon_{\theta} \left(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2 \right]$$

where $t \sim U(1, T)$